



机器狗精准控制

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程简述
- 三、实验环境准备 (0.5学时)
- 四、机器狗介绍 (0.5学时)
- 五、工程文件目录
- 六、精准控制 (4.5学时)
- 七、运行示例
- 八、附录

目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识1 UDP(用户数据协议, User Datagram Protocol)

1.6 背景知识2 机器狗运动正向求解

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

七、运行示例

八、附录

1.1 实验描述

本实验的目标是基于UDP协议精准控制机器狗的指令动作，准确控制狗的动作、参数等。实验使用的方式包含多种动作，如打招呼、扭身体、太空步、扭身跳等动作，还有对机器狗行走转弯路程的精准控制，同时在昇腾芯片上对多种动作对应的编码指令进行转换并实现相应动作。

目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识1 UDP(用户数据协议, User Datagram Protocol)

1.6 背景知识2 机器狗运动正向求解

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

七、运行示例

八、附录

1.2 实验目的

- 了解机器狗的运动逻辑、控制逻辑
- 熟悉机器狗的运动指令、运动参数
- 掌握运动数据的建模方式。
- 掌握机器狗的UDP协议发送。

目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识1 UDP(用户数据协议, User Datagram Protocol)

1.6 背景知识2 机器狗运动正向求解

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

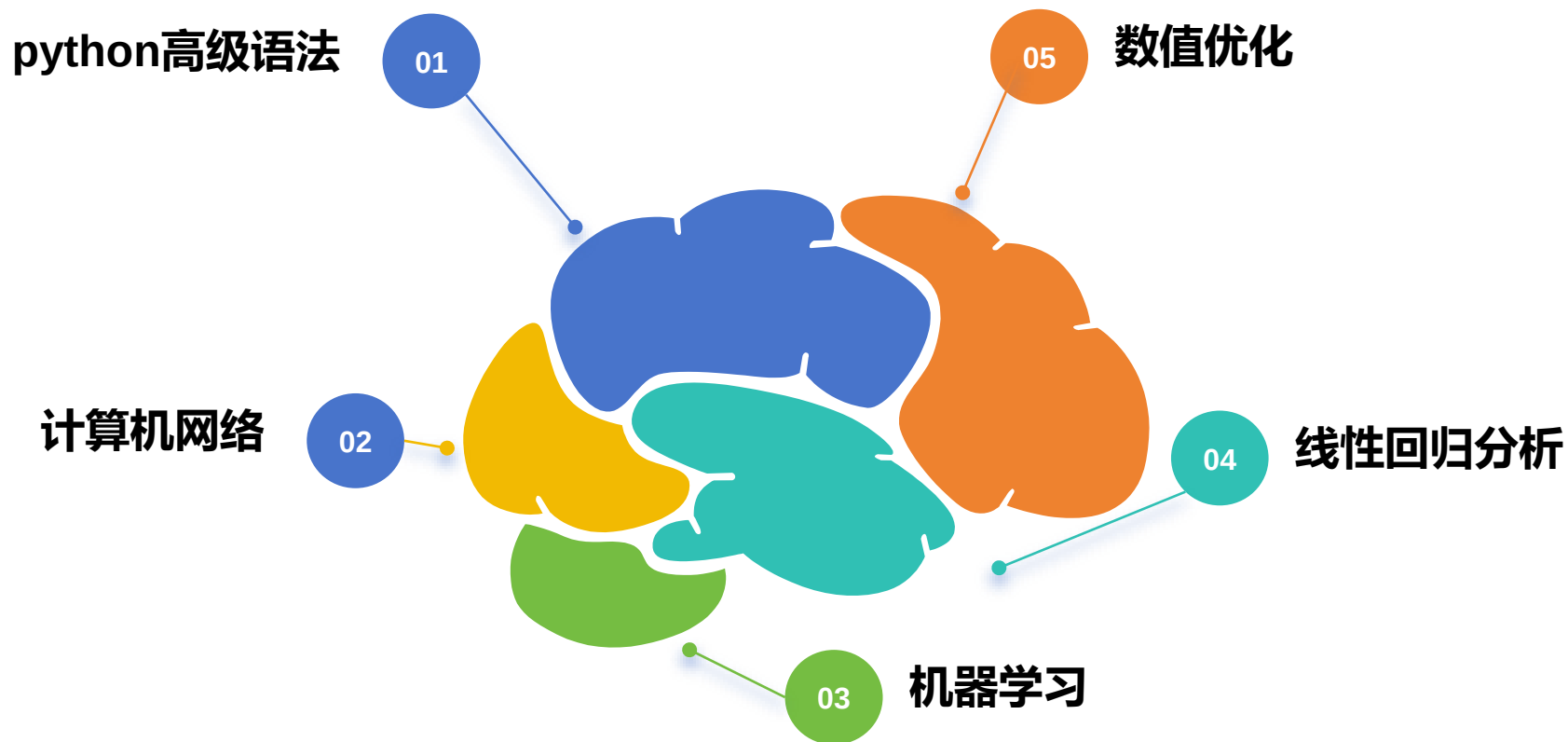
六、精准控制 (4.5学时)

七、运行示例

八、附录

1.3 实验建议

本实验主要是用 Python 编程，在开发板上进行运动模型的搭建，调优，最后需要在华为昇腾开发板上进行实际控制。



目录

一、实验前置准备 (0.5学时)

1.1 实验描述

1.2 实验目的

1.3 实验建议

1.4 实验环境

1.5 背景知识1 UDP(用户数据协议, User Datagram Protocol)

1.6 背景知识2 机器狗运动正向求解

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

七、运行示例

八、附录

1.4 实验环境——实验硬件设备

本实验需要的准备的硬件如下：（详见章节3.1、3.4）



1.5 软硬件环境准备

本实验需要的准备的软件环境如下：（详见章节3.2、3.3）

1



MobaXterm

2



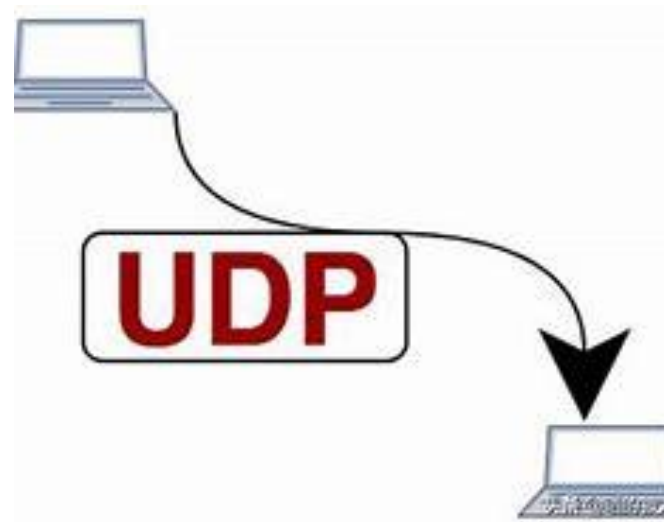
VSCode

1.5 背景知识1 UDP（用户数据报协议）

一种无连接的网络传输协议，它属于传输层协议，与TCP（传输控制协议）并列，但两者在设计和用途上有所不同。

UDP 协议的主要特点如下：

- 1.无连接：UDP 是无连接的协议，发送数据之前不需要建立连接。它只是简单地将数据打包到数据报中并发送，不保证数据报的顺序和可靠性。
- 2.简单性：UDP 协议的实现相对简单，因为它没有复杂的控制机制，如TCP中的流量控制、拥塞控制和重传机制。



UDP在机器人中的应用：

- 1.实时控制：机器人控制通常要求实时响应，UDP由于其低延迟特性，非常适合实时控制系统。
- 2.传感器数据传输：机器人上的传感器产生的数据可以通过UDP快速发送到控制单元进行处理。

1.6 背景知识2 机器狗运动正向求解

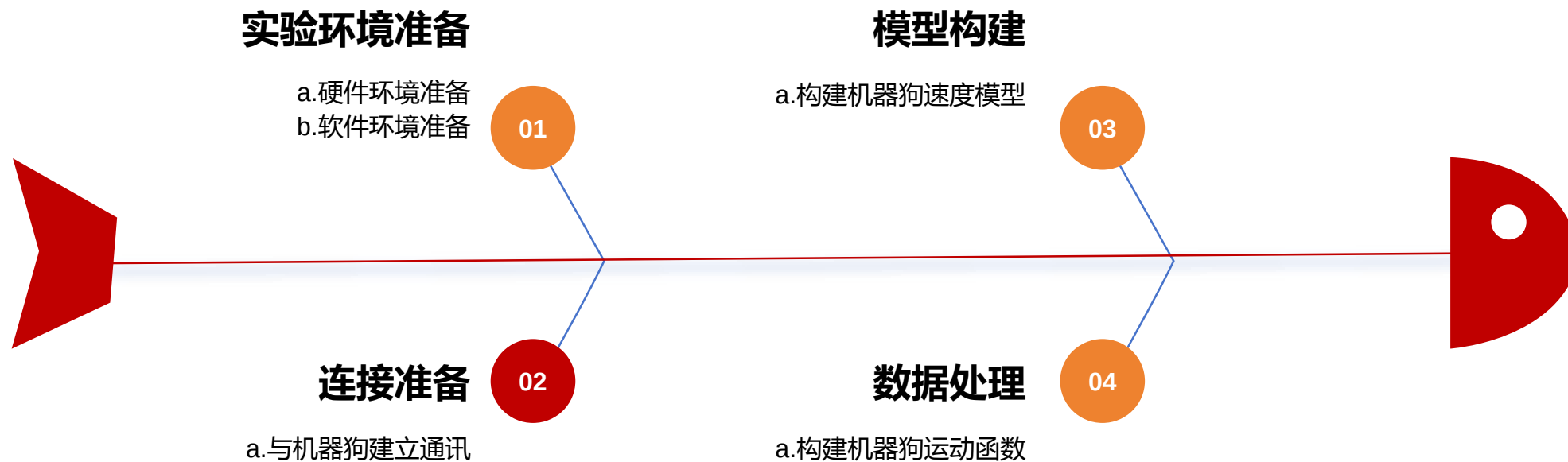
运动方程求解通常是指在已知物体的初始条件和作用力的情况下，使用物理定律来确定物体的运动状态（如位置、速度、加速度）的过程。以下是求解运动方程的一般步骤：

- a. 确定已知条件
- b. 选择合适的物理定律
- c. 建立运动方程
- d. 求解方程
- e. 考虑约束条件
- f. 模拟和验证
- g. 分析结果

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程简述
- 三、实验环境准备 (0.5学时)
- 四、机器狗介绍 (0.5学时)
- 五、工程文件目录
- 六、精准控制 (4.5学时)
- 七、运行示例
- 八、附录

2.实验过程简述



目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

3.1 硬件环境准备

3.2 机器狗远程连接

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

七、运行示例

八、附录

3.1 硬件环境准备

(1) SD卡中系统烧录

向SD卡中烧录镜像的步骤如下：

- ① 将SD卡插入读卡器，再将读卡器插到PC上。



3.1 硬件环境准备

(1) SD卡中系统烧录

②在浏览器中进入Ascend官网，选择“产品”->“开发者套件”



3.1 硬件环境准备

(1) SD卡中系统烧录

③点击“资源”，进入资源下载界面



3.1 硬件环境准备

(1) SD卡中系统烧录

④选择“制卡工具下载”，下载完成后进行安装。

寻你所需，玩转AI开发_

快速入门

准备

请提前做好下列物品，或查看 [硬件准备清单](#)，快速上手不卡顿

- MicroSD卡（推荐64GB）及读卡器（可选）
- 获取制卡工具，一键烧录镜像完成开发环境准备

[制卡工具下载](#) [校验文件下载](#)

入门

提供入门 Atlas 200I DK A2 的全流程指导，帮助你30分钟内轻松完成从启动开发者套件到运行首样例的过程

[快速开始（课程）](#) [快速开始（文档）](#)

参考工具

提供Atlas 200I DK A2 进一步开发所需的参考工具及配套资源，可按需下载

模型适配工具

使能开发者0代码完成数据标注、模型训练、模型部署到昇腾开发者套件

[软件包下载](#) [校验文件下载](#)

配套Conda环境包

模型适配工具运行所必须的配套conda依赖环境，需要提前安装conda并导入conda环境包

[软件包下载](#) [校验文件下载](#)

3.1 硬件环境准备

(1) SD卡中系统烧录

⑤打开镜像烧录工具Ascend AI Devkit Imager, 选择Atlas 200I DK A2



3.1 硬件环境准备

(1) SD卡中系统烧录

⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到Type-C接口默认IP为192.168.0.2。选择默认IP。

镜像网络信息配置

×

以下为镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网口信息

ETH0

ETH1

Type-C

☐ 自动获取IP地址

☒ 使用下面的IP地址

IP地址

192 · 168 · 0 · 2

子网掩码

255 · 255 · 255 · 0

默认网关

· · ·

首选DNS服务器

8 · 8 · 8 · 8

备用DNS服务器

114 · 114 · 114 · 114

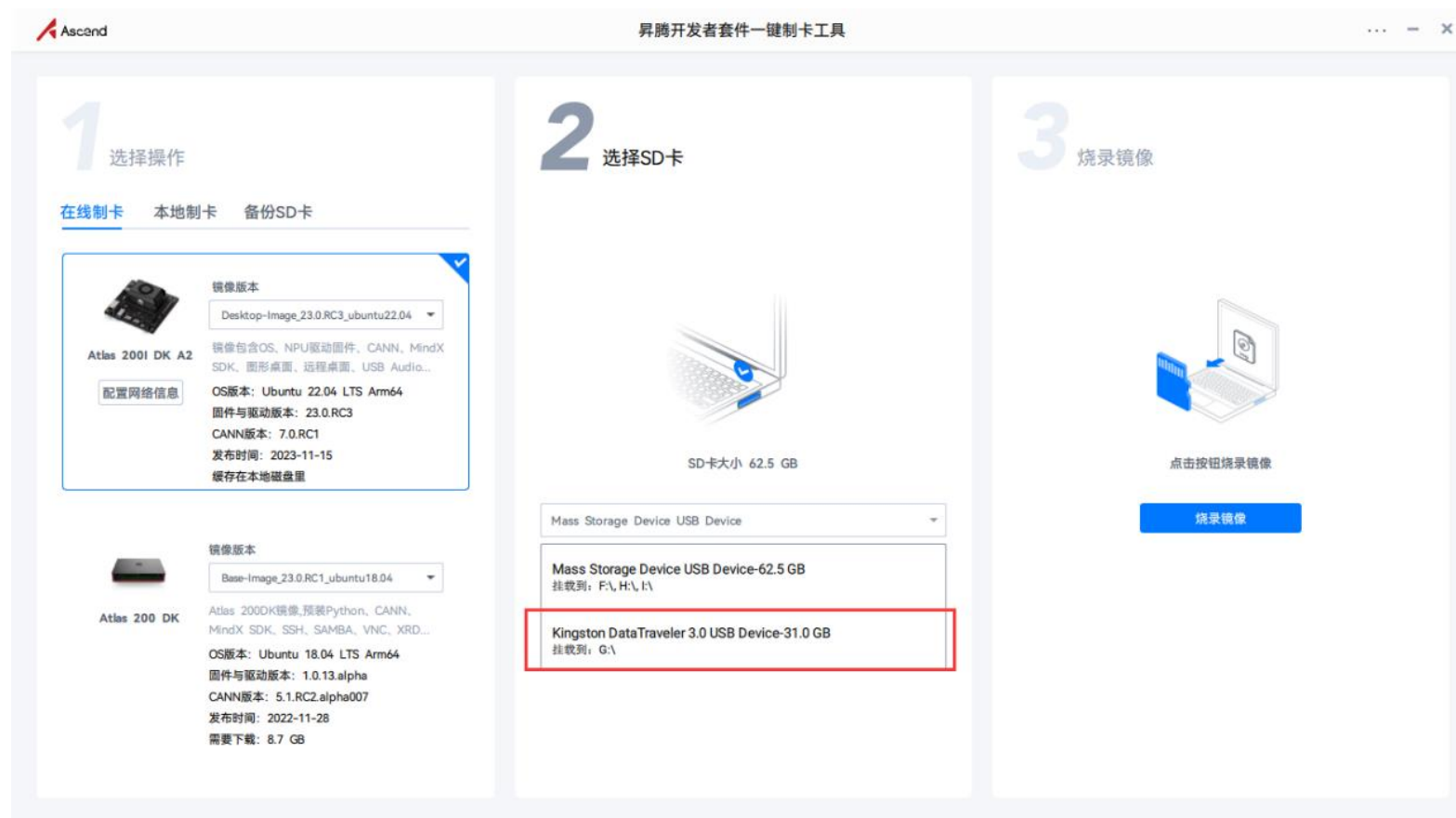
确定

取消

3.1 硬件环境准备

(1) SD卡中系统烧录

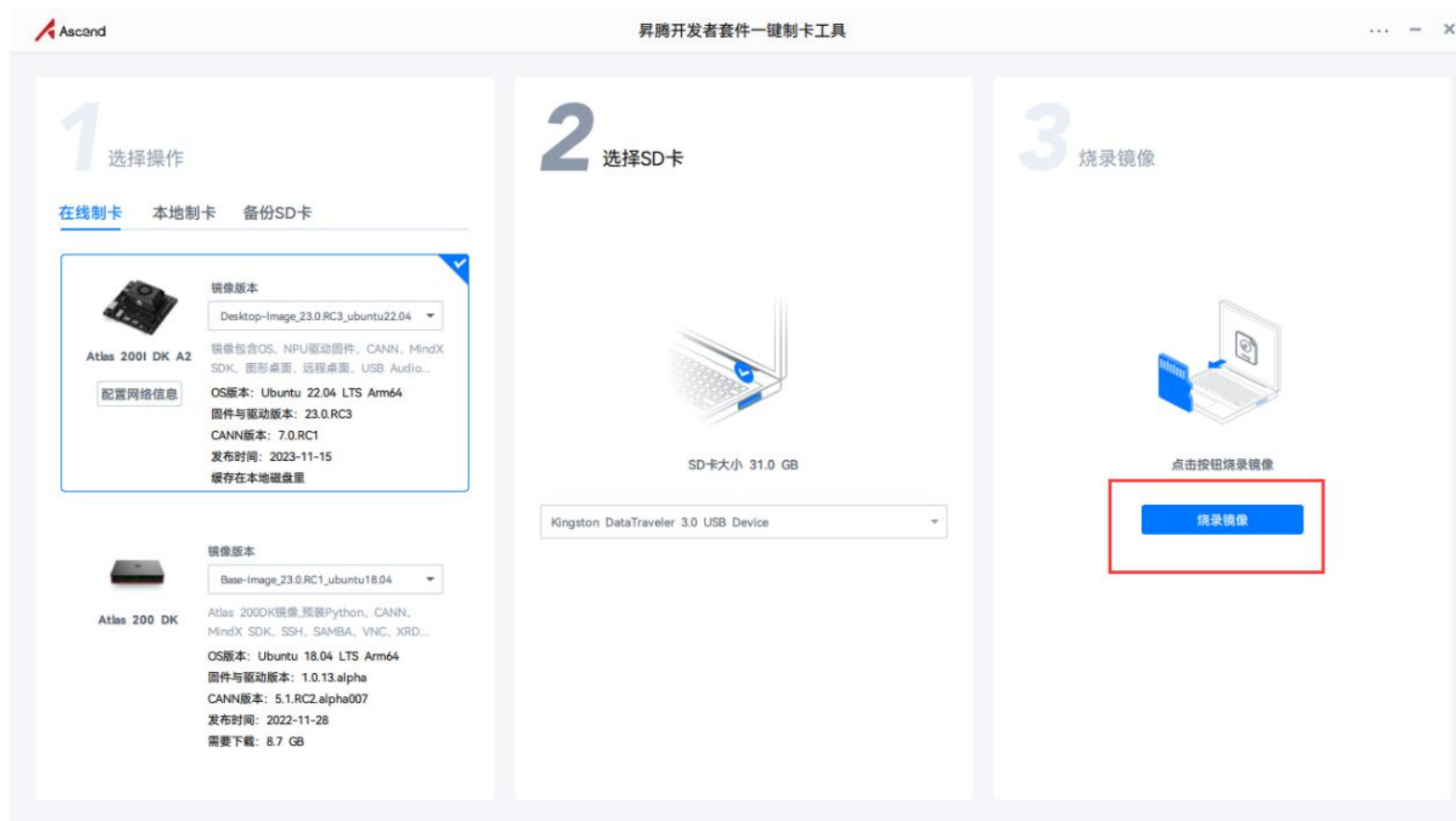
⑦选择要烧录的SD卡。



3.1 硬件环境准备

(1) SD卡中系统烧录

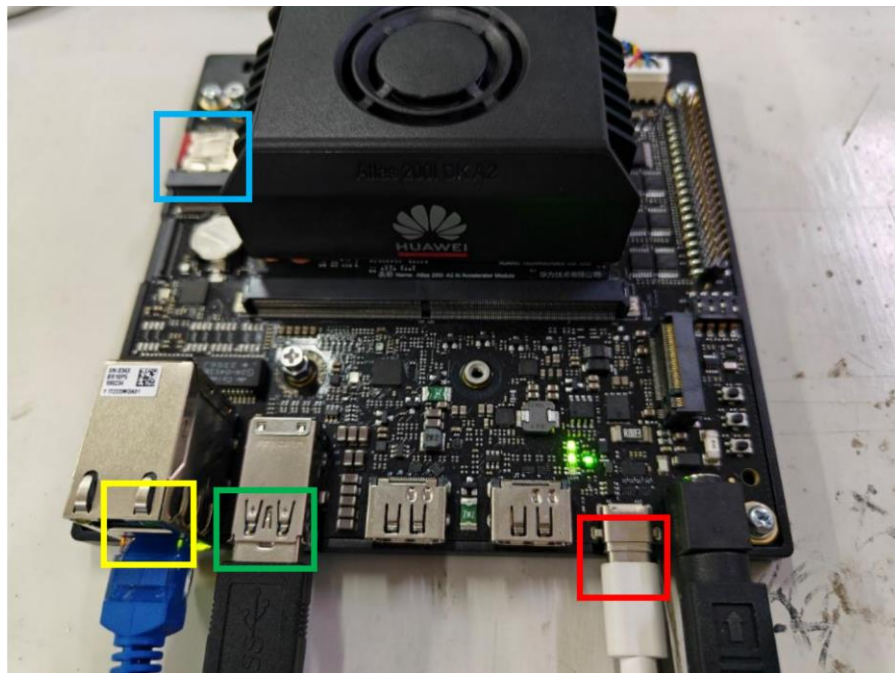
⑧点击“烧录镜像”，等待镜像烧录完成。



3.1 硬件环境准备

(2) 开发板连接

将烧录好系统的SD卡插入到开发板中（下方蓝色框中），昇腾开发板和PC机通过Type-C线连接，将Type-C线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的USB线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



3.1 硬件环境准备

(3) PC端远程连接开发板

当开发者套件通过Type-C接口和PC连接时，如何在PC将接口 IP地址修改为和开发者套件接口 IP地址同一个网段（如开发者套件Type-C 接口为192.168.0.2， PC对应USB或Type-C接口为192.168.0.101）， 并使用SSH工具远程登录开发者套件。

1) 安装Windows的USB网卡驱动（以Windows 10系统为例）

①在PC上右键单击“此电脑”，选择“更多”→“管理”，进入“计算机管理”界面。

②在“计算机管理”操作界面中选择“设备管理器→“其他设备”，如图所示，RNDIS为未识别状态。

注意：如果未出现RNDIS，请按以下方法排查：

确保开发者套件Type-C接口和PC已通过连接线正常连接。

更换具备数据传输能力的Type-C数据线（一般手机充电线拥有数据传输能力），继续查看是否出现RNDIS。

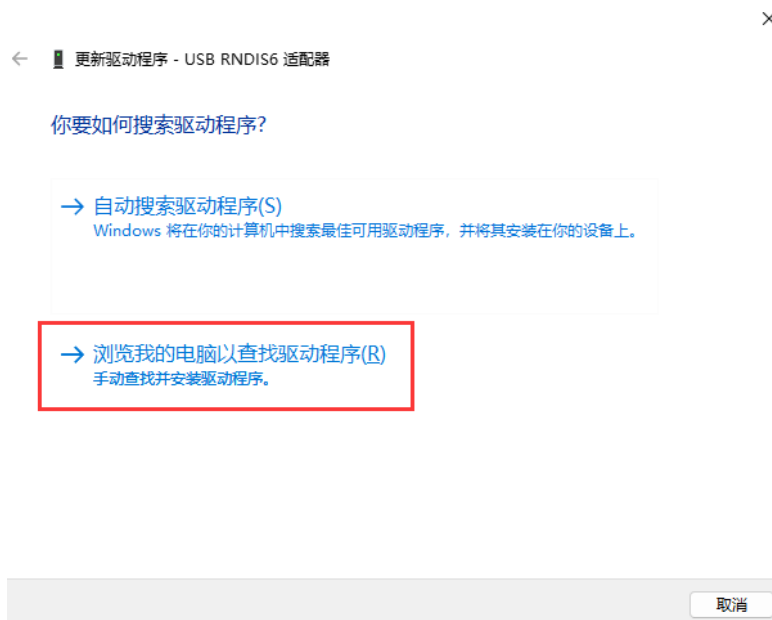
3.1 硬件环境准备

(3) PC端远程连接开发板

如果尝试现场所有Type-C数据线，仍无法出现RNDIS，则考虑使用 RJ45网线连接PC和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

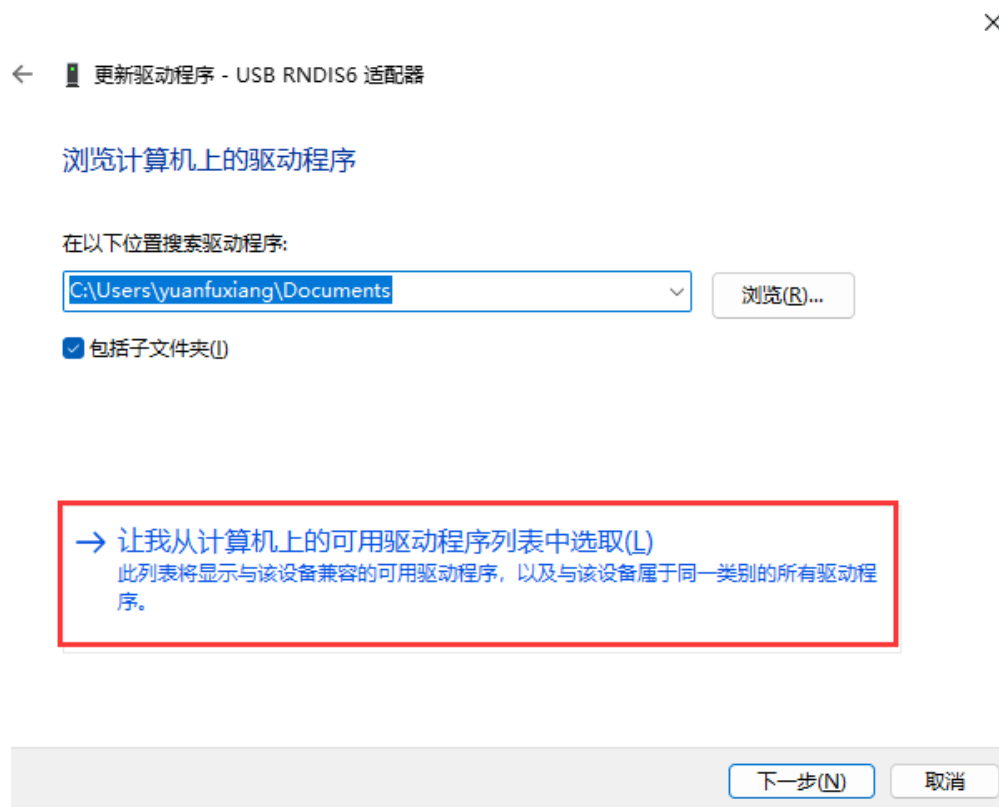
④在弹出的“更新驱动程序”→“RNDIS窗口”中选择“浏览我的计算机以查找驱动程序软件(R)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

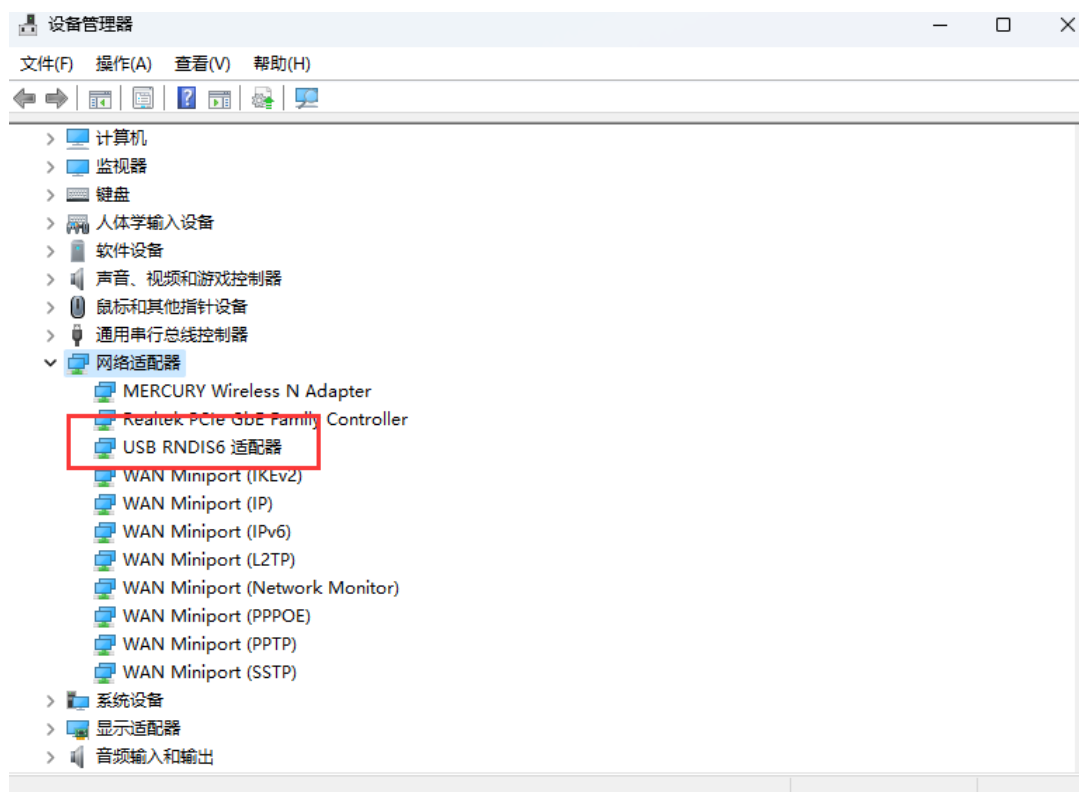
⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6适配器”。



3.1 硬件环境准备

(3) PC端远程连接开发板

- ⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。
- ⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了USB RNDIS6适配器的驱动。



3.1 硬件环境准备

(4) 配置PC接口IP地址

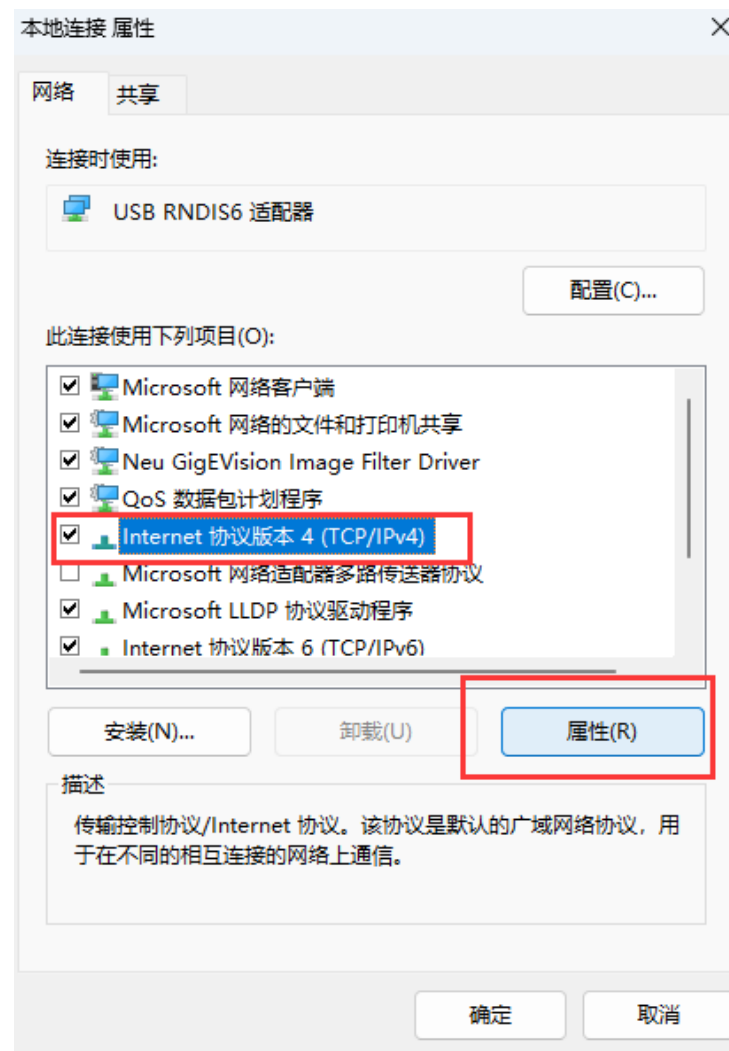
本步骤以 Windows 10 系统为例。

- ①在 PC 上打开 “开始” ， 单击 “设置” 按钮， 进入 “Windows 设置” 界面。
- ②选择 “网络和 Internet” ， 单击 “更改适配器选项” 。
- ③鼠标右键单击 “本地连接” 后鼠标左键单击 “属性” 进入 “本地连接 属性” 界面（使用 Type-C 接口连接时一般为 “本地连接 x” ， x 为数字， 以实际显示的数字为准。

3.1 硬件环境准备

(4) 配置PC接口IP地址

④选择 “Internet协议版本4 (TCP/IPv4) ” ， 单击 “属性” 。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑤勾选“使用下面的IP地址”选项，填写IP地址（图示以192.168.0.101为例）和子网掩码，默认网关与DNS服务器地址为空，单击“确定”保存。

Internet 协议版本 4 (TCP/IPv4) 属性

常规

如果网络支持此功能，则可以获取自动指派的 IP 设置。否则，你需要从网络系统管理员处获得适当的 IP 设置。

☐ 自动获得 IP 地址(O)

☒ 使用下面的 IP 地址(S):

IP 地址(I): 192 . 168 . 0 . 101

子网掩码(U): 255 . 255 . 255 . 0

默认网关(D): . . .

☐ 自动获得 DNS 服务器地址(B)

☒ 使用下面的 DNS 服务器地址(E):

首选 DNS 服务器(P): . . .

备用 DNS 服务器(A): . . .

☐ 退出时验证设置(L)

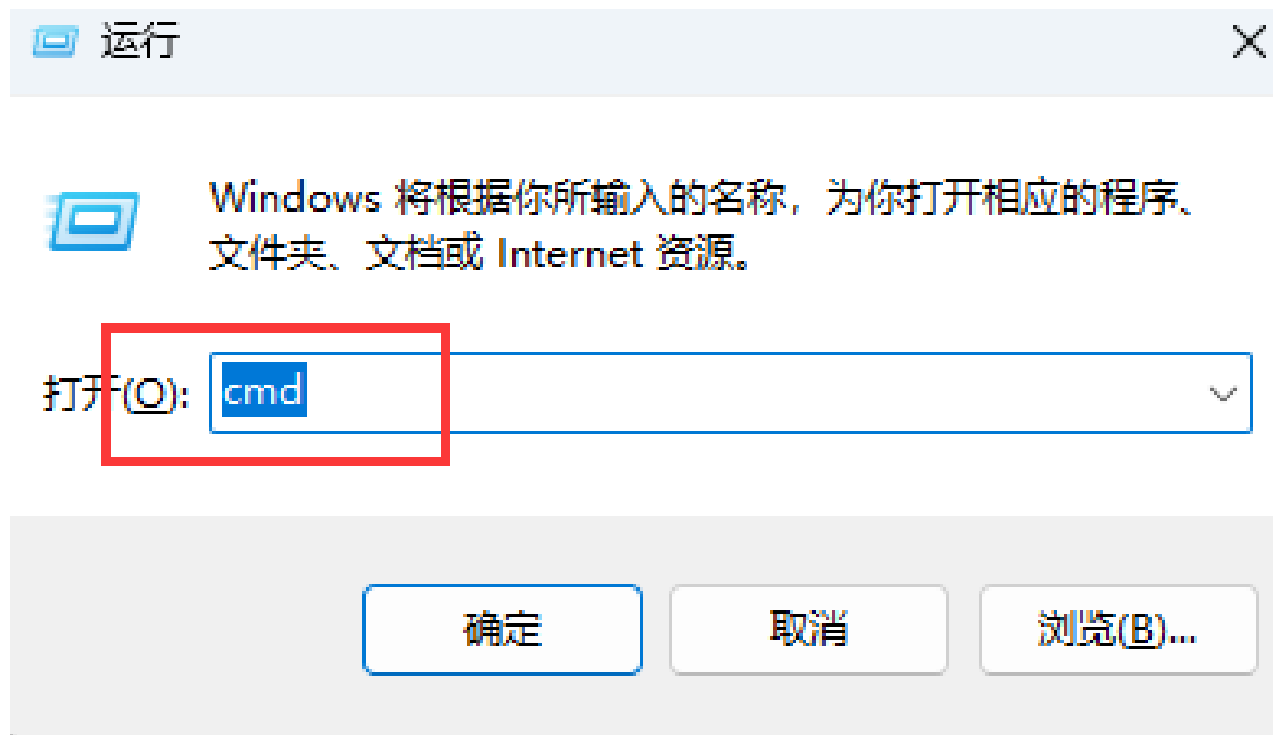
高级(V)...

确定 取消

3.1 硬件环境准备

(4) 配置PC接口IP地址

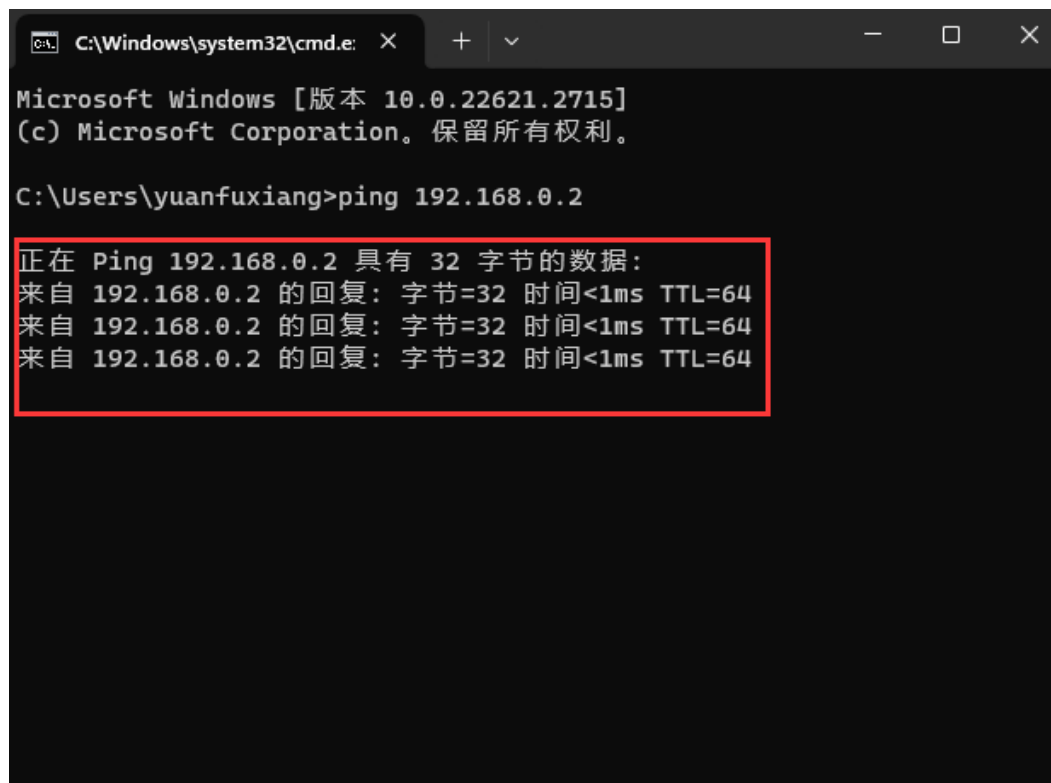
⑥使用快捷键 “Win+R” ， 在运行窗口输入cmd进入命令行窗口。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑦输入ping 192.168.0.2命令，查看是否能够访问开发板IP。



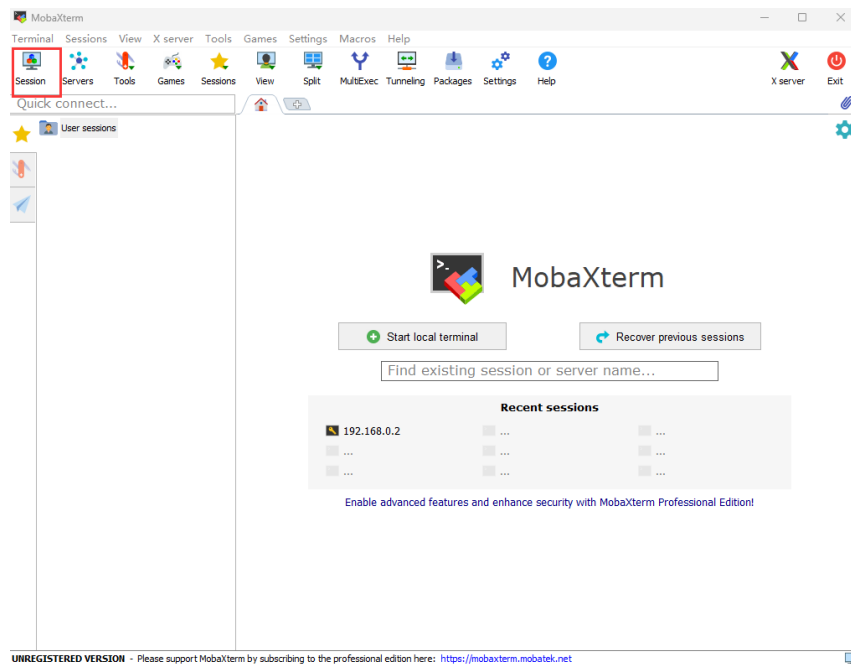
```
C:\Windows\system32\cmd.exe X + - □ X  
Microsoft Windows [版本 10.0.22621.2715]  
(c) Microsoft Corporation。保留所有权利。  
C:\Users\yuanfuxiang>ping 192.168.0.2  
正在 Ping 192.168.0.2 具有 32 字节的数据:  
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64  
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64  
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
```

3.1 硬件环境准备

(5) 使用ssh远程登录开发板

本文使用MobaXterm工具在PC上远程登录开发板，单击下载链接获取MobaXterm软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

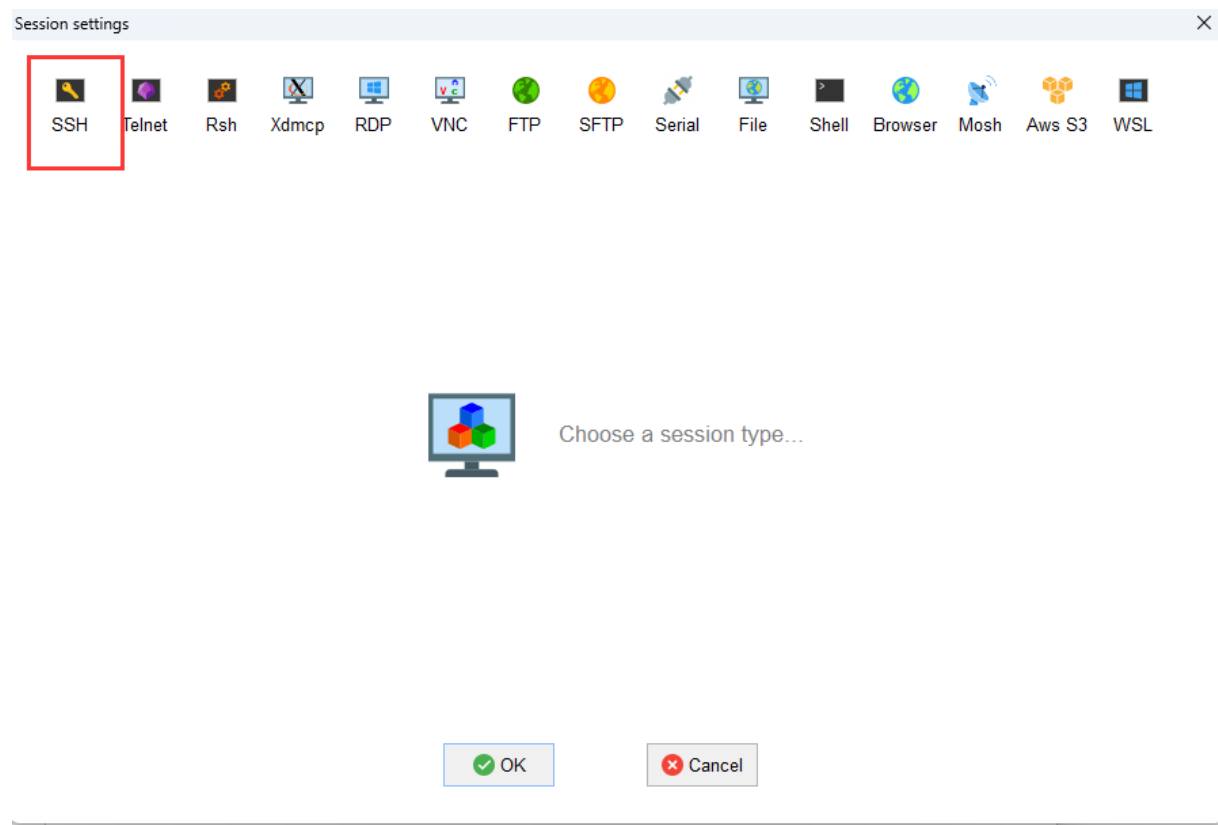
- ①双击“MobaXterm_Personal_22.2.exe”启动SSH登录工具。
- ②单击左上方的“Session”进入界面。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

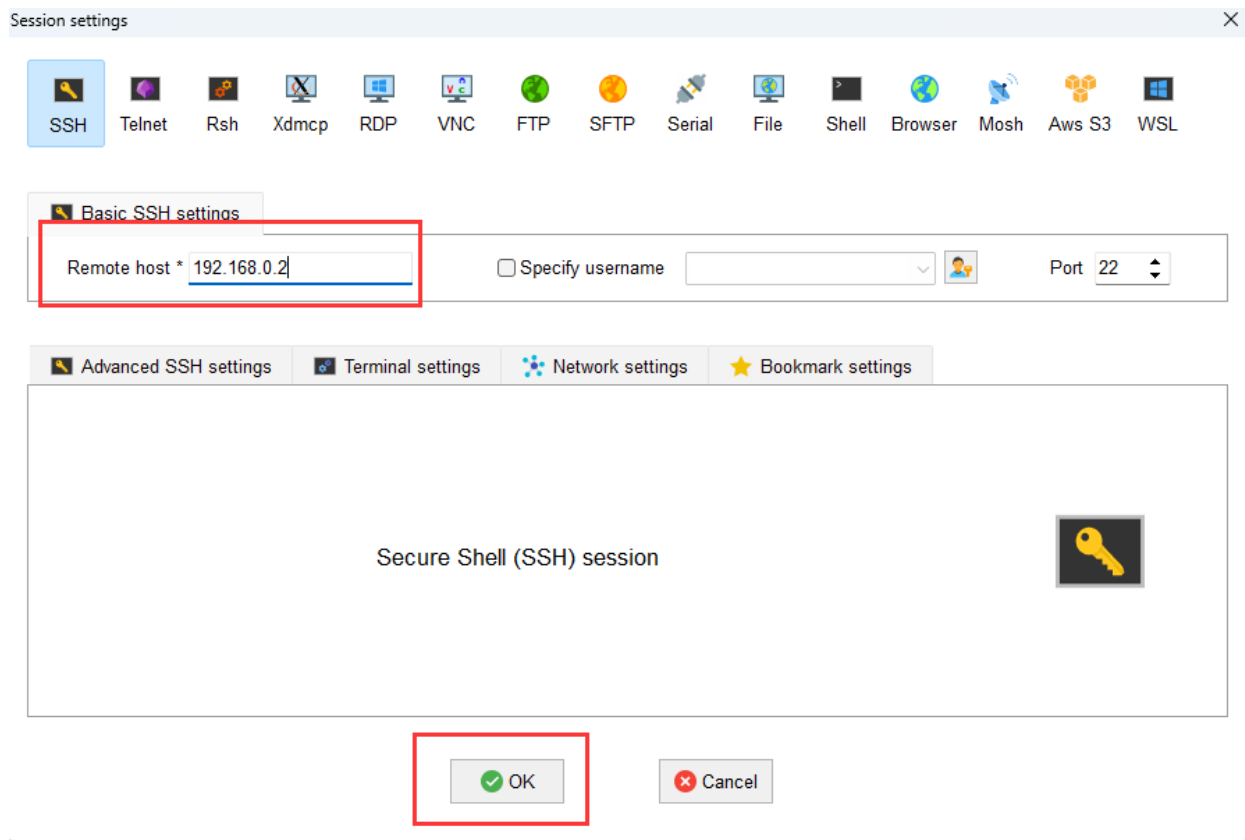
③单击左上方的“SSH”进入 SSH 连接配置界面



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

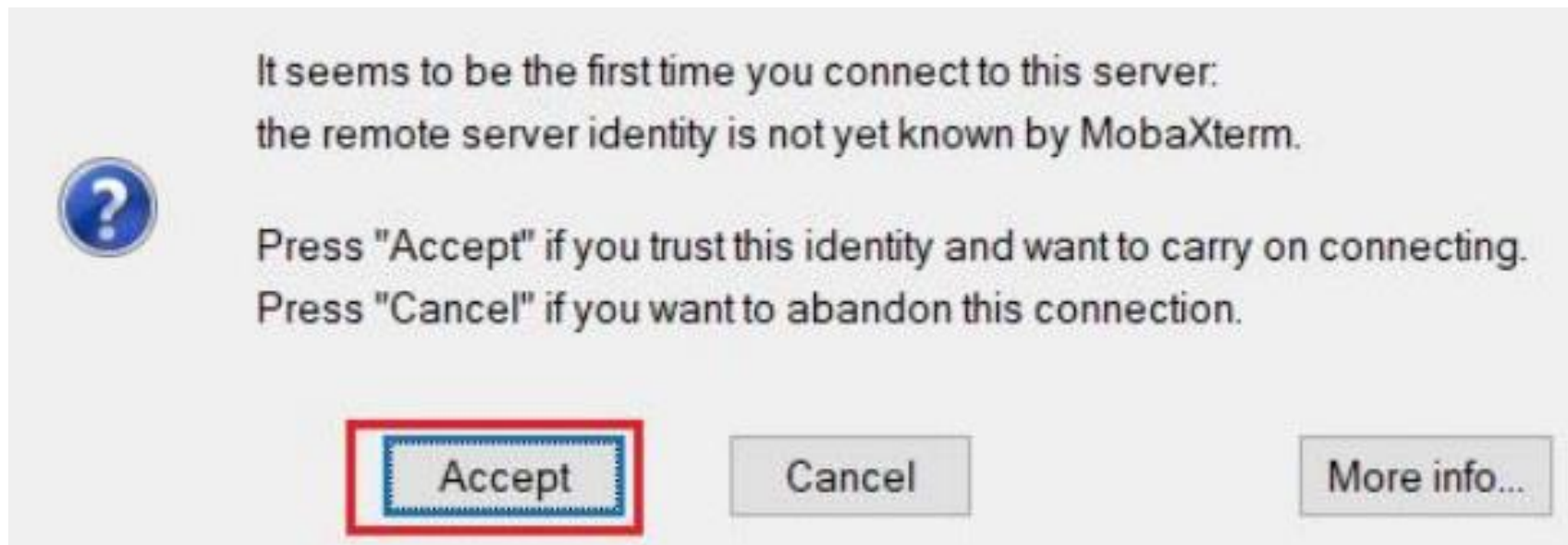
④根据硬件连接方式填写开发者套件连接PC的接口实际IP地址（一键制卡中配置的IP地址）。图示以192.168.0.2为例，请根据实际修改。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

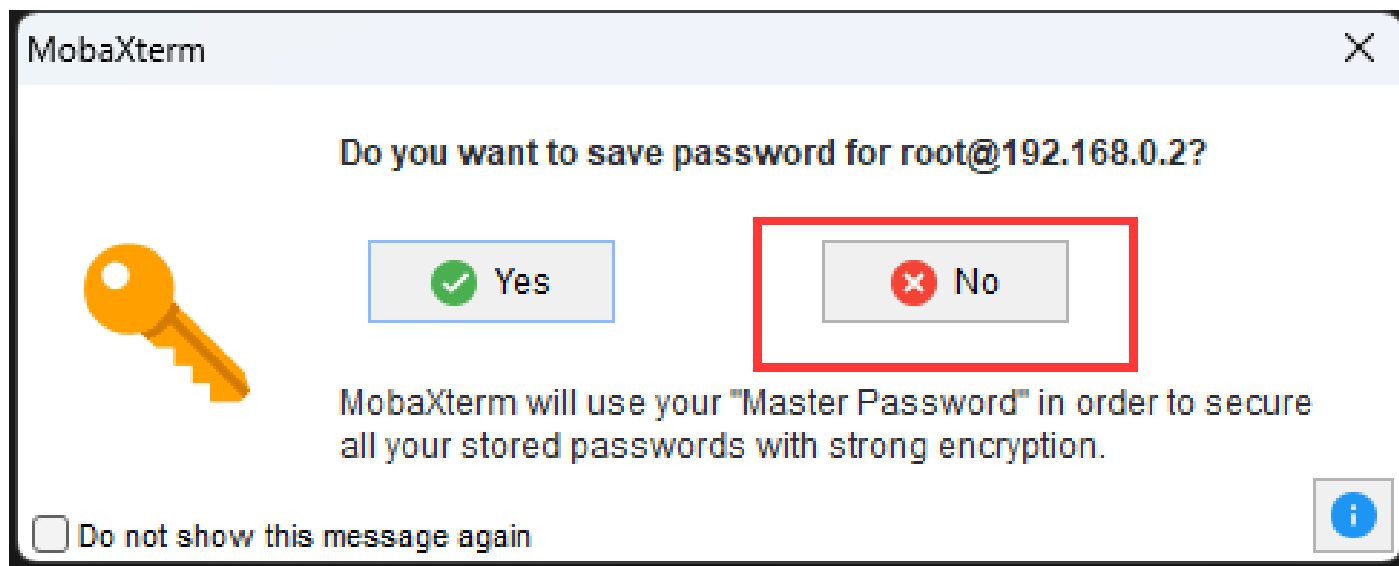
⑤单击“OK”按钮，首次连接开发者套件时，SSH工具提示是否信任连接的设备，单击“Accept”。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

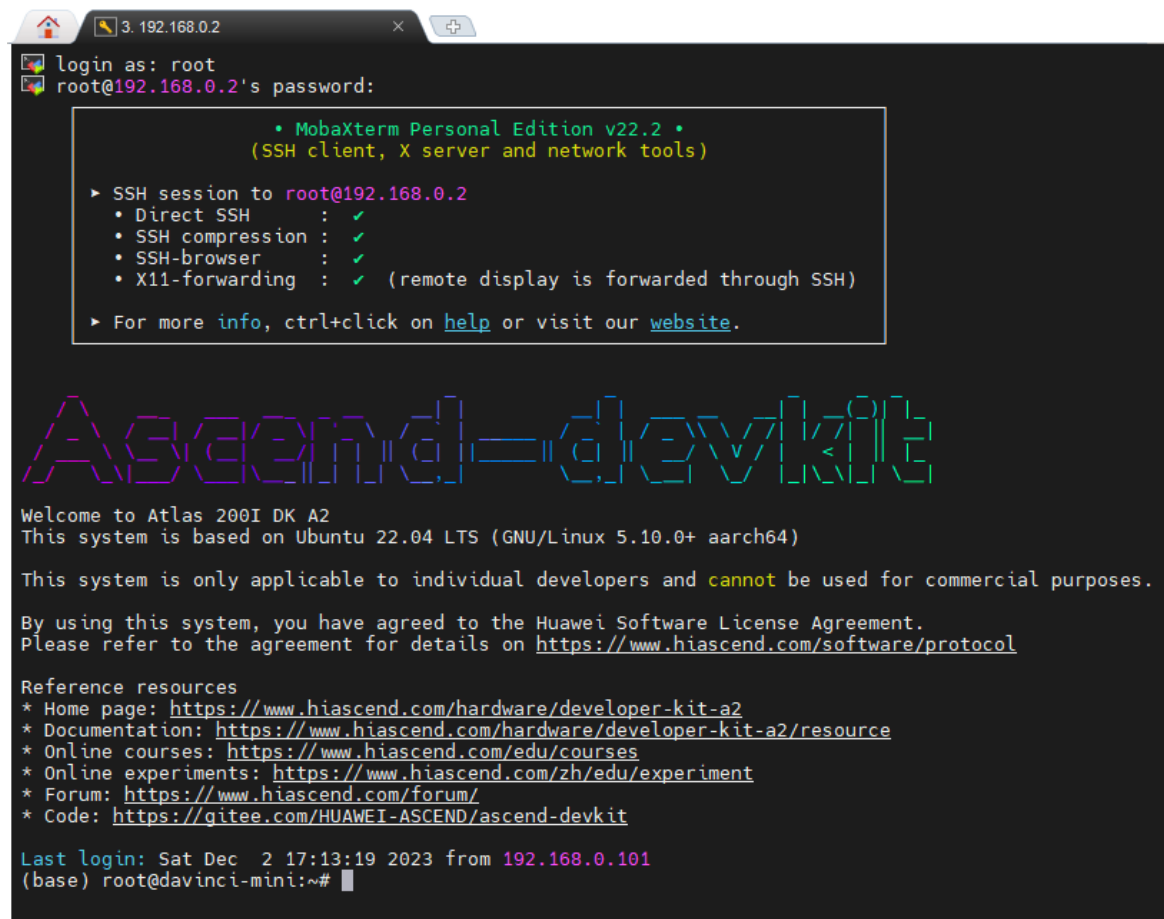
⑥进入远程登录界面后，输入 root 用户名登录密码（默认为 Mind@123）登录开发者套件，请修改默认密码，并妥善保管新密码。SSH 工具界面会出现保存密码提示，可以单击 “No”，不保存密码直接登录开发者套件。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

⑦远程登录开发者套件成功界面如图所示。



```
3. 192.168.0.2
login as: root
root@192.168.0.2's password:
MobaXterm Personal Edition v22.2
(SSH client, X server and network tools)
SSH session to root@192.168.0.2
Direct SSH : ✓
SSH compression : ✓
SSH-browser : ✓
X11-forwarding : ✓ (remote display is forwarded through SSH)
For more info, ctrl+click on help or visit our website.

Ascend=devkit

Welcome to Atlas 200I DK A2
This system is based on Ubuntu 22.04 LTS (GNU/Linux 5.10.0+ aarch64)

This system is only applicable to individual developers and cannot be used for commercial purposes.

By using this system, you have agreed to the Huawei Software License Agreement.
Please refer to the agreement for details on https://www.hiascend.com/software/protocol

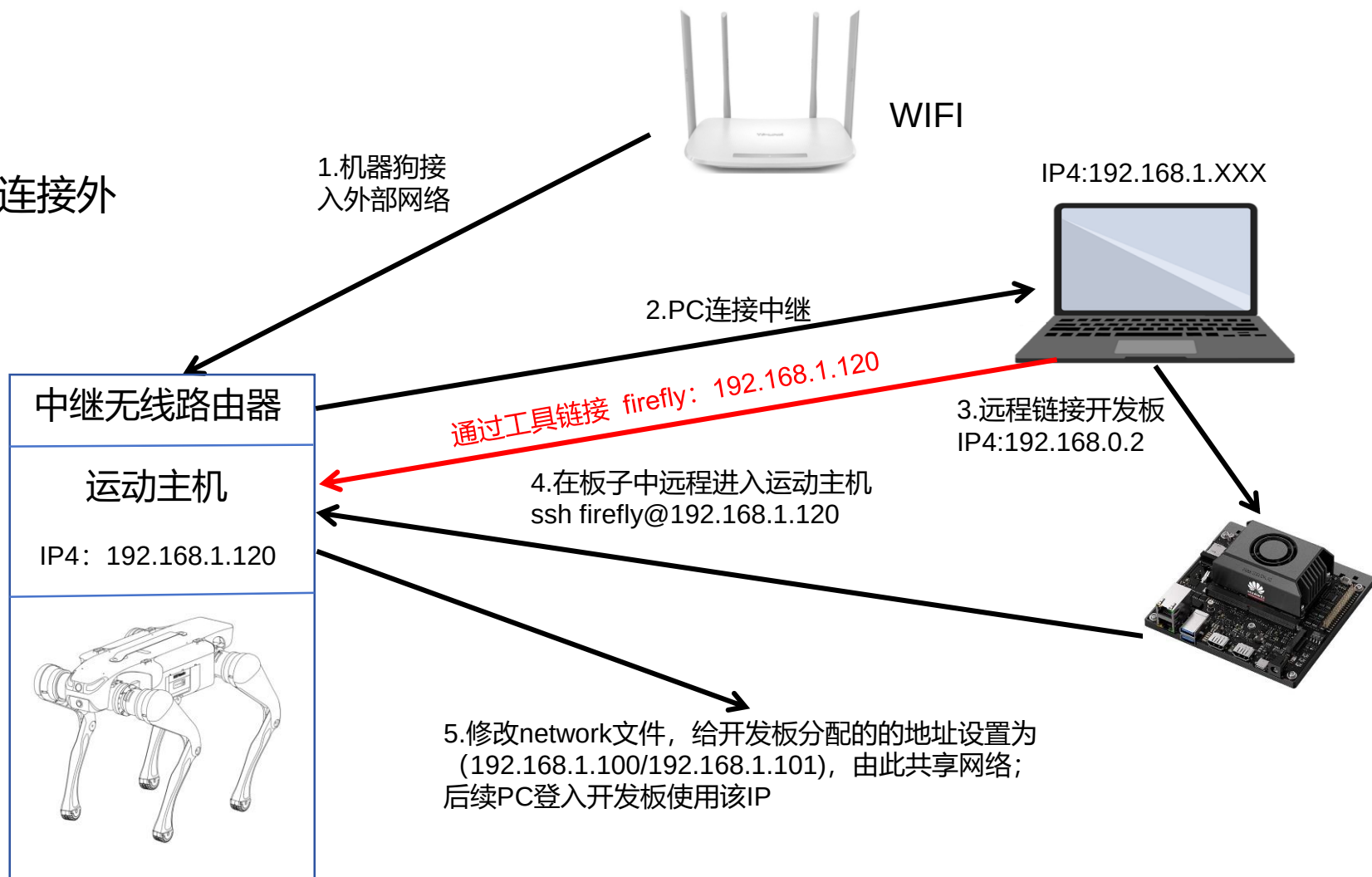
Reference resources
* Home page: https://www.hiascend.com/hardware/developer-kit-a2
* Documentation: https://www.hiascend.com/hardware/developer-kit-a2/resource
* Online courses: https://www.hiascend.com/edu/courses
* Online experiments: https://www.hiascend.com/zh/edu/experiment
* Forum: https://www.hiascend.com/forum/
* Code: https://gitee.com/HUAWEI-ASCEND/ascend-devkit

Last login: Sat Dec 2 17:13:19 2023 from 192.168.0.101
(base) root@davinci-mini:~#
```

3.1 硬件环境准备

(6) 机器狗中继连接外网

此步骤是为了让单机环境变为连接外部网络的环境



3.1 硬件环境准备

(6) 机器狗中继连接外网

注：本教程主要为暂时没有外置无线网卡的用户提供连接外网方案，教程以中继手机热点为例进行说明，中继无线路由器同理。

1、电脑连接 机器狗的wifi，点击电脑右下角的wifi图标，点击属性查看IPv4 DNS服务器的ip地址。如下图所示。



属性	
SSID:	YSC-JYML-emzqzj
协议:	Wi-Fi 4 (802.11n)
安全类型:	WPA2-个人
网络频带:	2.4 GHz
网络通道:	11
链接速度(接收/传输):	144/144 (Mbps)
IPv4 地址:	192.168.1.101
IPv4 DNS 服务器:	192.168.1.254 8.8.8.8
制造商:	Intel Corporation
描述:	Intel(R) Wireless-AC 9560 160MHz
驱动程序版本:	22.200.0.6
物理地址(MAC):	
复制	

3.1 硬件环境准备

(6) 机器狗中继连接外网

2、打开电脑浏览器，输入IPv4 DNS服务器的ip，默认登录用户和密码均为admin。
如下图所示。

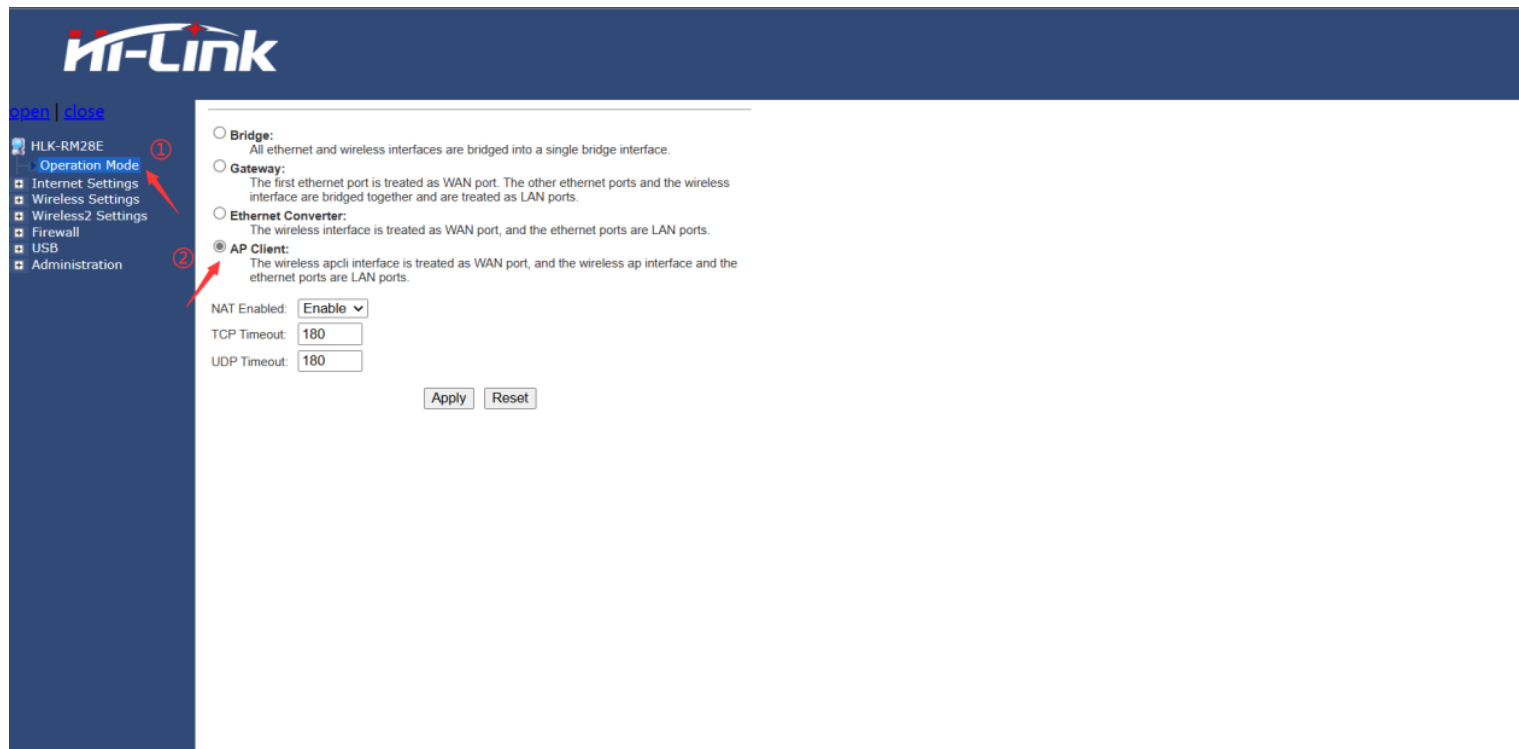


3.1 硬件环境准备

(6) 机器狗中继连接外网

3、打开手机热点。

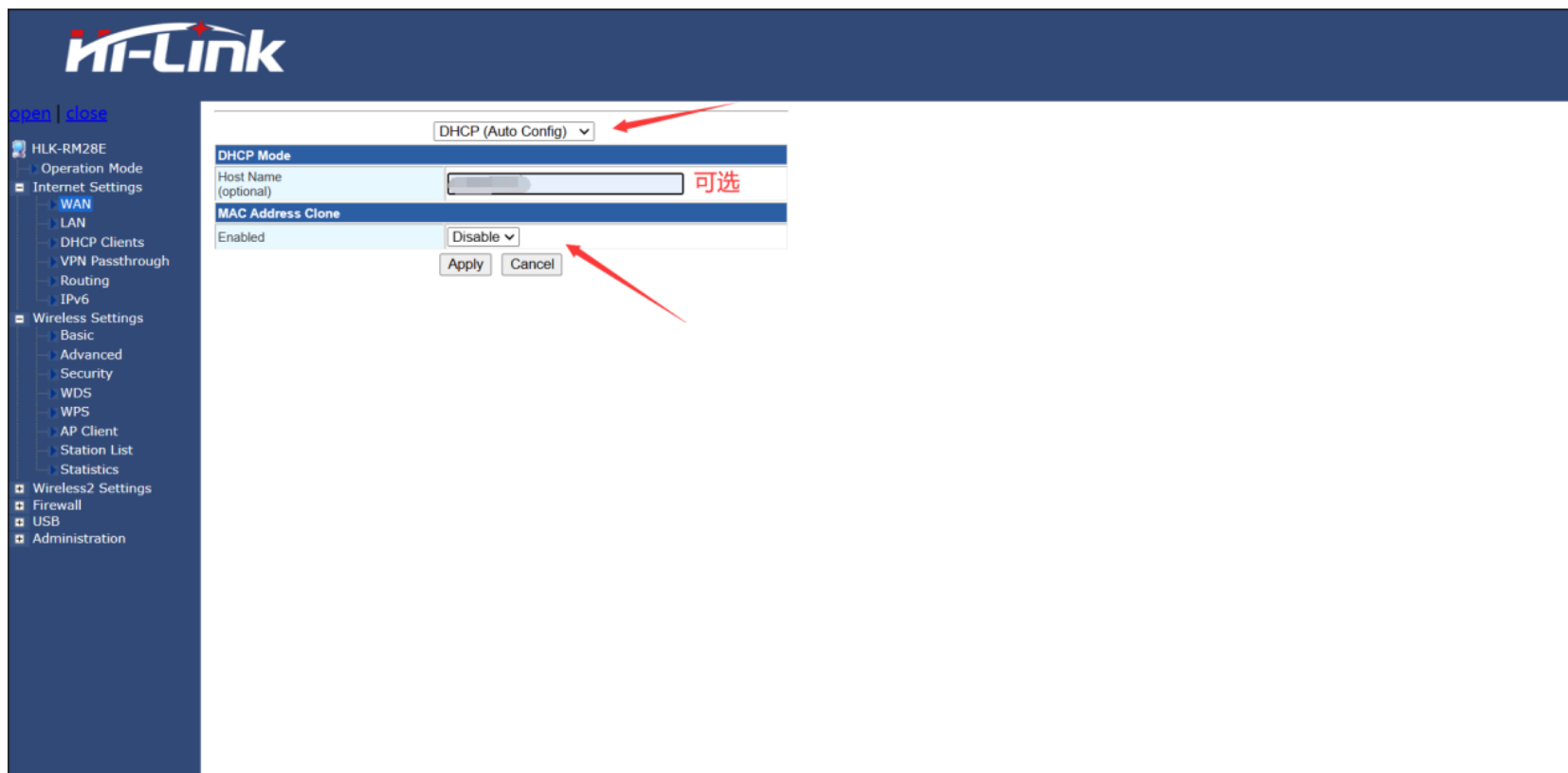
4、进入Hi-link设置页面，点击Operation Mode，选择AP Client，最后点击Apply。如下图所示。



3.1 硬件环境准备

(6) 机器狗中继连接外网

5、展开Internet Settings，点击WAN，参照下图进行设置，最后点击Apply。



3.1 硬件环境准备

(6) 机器狗中继连接外网

6、展开Wireless Settings，点击AP Client，查看site Survey表中是否有自己的手机热点名称，如若没有点击SCAN进行扫描，在AP client Parameters表中参照下图进行设置，最后点击Apply。

HLK-RM28E

Operation Mode

Internet Settings

Wireless Settings

- Basic
- Advanced
- Security
- WDS
- WPS
- AP Client
- Station List
- Statistics

Wireless2 Settings

Firewall

USB

Administration

AP Client Feature

You could configure AP Client parameters here.

AP Client Parameters

SSID

HelloWorld

MAC Address (Optional)

Security Mode

WPA2PSK

Encryption Type

AES

Pass Phrase

Apply

Cancel

SCAN

Site Survey

Ch	SSID	BSSID	Security	Signal(%)	W-Moe	ExtCh	NT
1	weichao	f4 84 8d 97 88 fc	WPA1PSKWPA2PSK/AES	68	11b/g/n	ABOVE	In
1	weichao	f4 84 8d 97 8f c2	WPA1PSKWPA2PSK/AES	70	11b/g/n	ABOVE	In
1	KRN1303	10 5d dc 2f a4 20	WPA1PSKWPA2PSK/AES	50	11b/g/n	NONE	In
1	iTV-X1oT	aa 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	jsscsc	ec 60 73 e4 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1		ee 60 73 74 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1	0xE997B9E5A587E69CBAE599A8E4BABA	a2 2e 55 9a b0 38	WPA1PSKWPA2PSK/AES	100	11b/g/n	NONE	In
1	ChinaNet-X1oT	9a 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	weichao	f4 84 8d 97 8b ab	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
3	Baiyuan	b8 f0 b9 53 b0 f8	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
4	409-DT-2G	80 3f 5d a9 ac 7e	WPA2PSK/AES	65	11b/g/n	ABOVE	In
4	ChinaNet-wmwX	b0 b1 94 ef d2 c3	WPAPSK/TKIPAES	68	11b/g/n	NONE	In
5	ChinaNet-zu2d	14 30 04 00 5d 84	WPA1PSKWPA2PSK/TKIPAES	63	11b/g/n	NONE	In
6	ChinaNet-WrQ4	2c 1a 01 d1 ea 88	WPA1PSKWPA2PSK/TKIPAES	78	11b/g/n	NONE	In
6	weichao	f4 84 8d 97 8e cc	WPA1PSKWPA2PSK/AES	86	11b/g/n	BELOW	In
6		18 3c b7 37 74 99	WPA2PSK/AES	94	11b/g/n	NONE	In

① 输入手机的热点名称

② 可选

③ 输入热点加密类型

④ 输入手机的热点密码

3.1 硬件环境准备

(6) 机器狗中继连接外网

7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

3.1 软硬件环境准备

3.2 机器狗远程连接

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

七、运行示例

八、附录

3.2 机器狗远程连接

用户可通过SSH 远程连接到运动主机。

将开发主机连接到
机器人WiFi。

远程连接运动主机

在开发主机上打开SSH 连
接软件

输入ssh
[firefly@192.168.1.120](#)
密码为 firefly,
或者直接通过SSH软件进入主
机

进入jy_exe, 并打
开配置文件

```
cd  
/home/firefly/jy_ex  
e/conf/
```

```
vim network.toml
```

配置文件
network.toml 内容

```
ip =  
'192.168.1.102'  
target_port = 43897  
local_port = 43893
```

修改配置文件

如果代码在机器人运动主机
内运行, IP 设置为运动主
机IP: 192.168.1.120;

如果代码在用户自己的开发主
机中运行, 设置为开发主机的
静态IP: 192.168.1.xxx;

重启运动程序

```
cd  
/home/firefly/jy_e  
xe  
sudo ./stop.sh  
sudo ./restart.sh
```

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

4.1研究背景与意义

五、工程文件目录

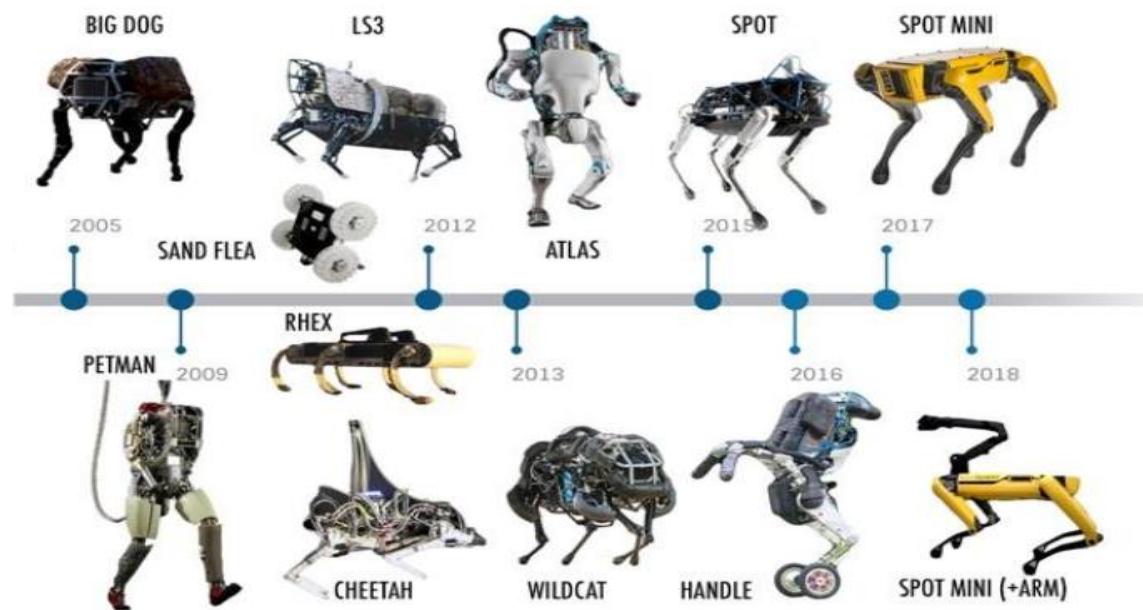
六、精准控制 (4.5学时)

七、运行示例

八、附录

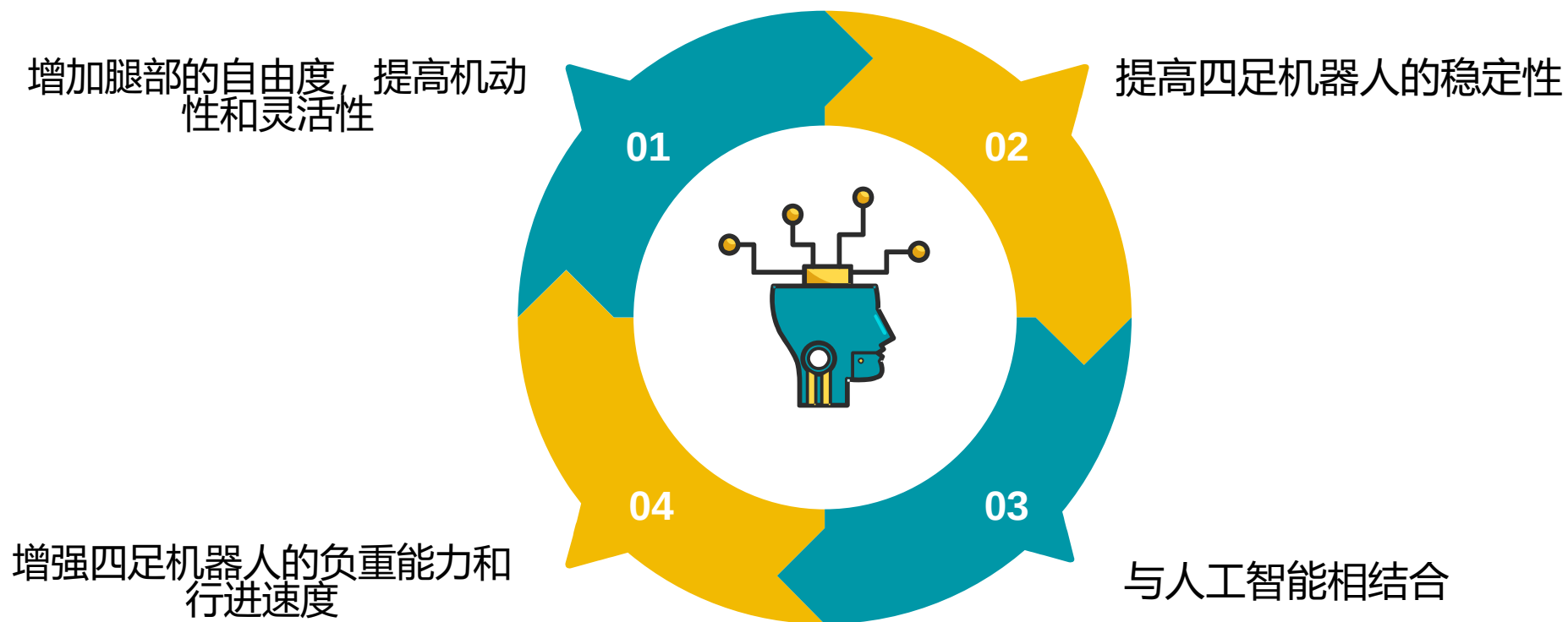
4.1 研究背景与意义

随着科技进步，社会生产力迅猛发展，机器人行业得到了蓬勃发展，其中四足机器人发展迅速，被广泛应用。传统的机器人如轮式、履带式，虽然具有很快的移动速度，但受地形约束。四足机器人是一种仿生机器人，在跨越地形障碍方面具有很大优势，足式移动机器人对行走路面的要求很低，它可以跨越障碍物，在各种崎岖不平的复杂路面上行进。在户外，车辆或履带式机器人无法应对大部分的复杂崎岖地形，四足式机器人以其灵活机动的特性在野外勘测和信息传递方面发挥重要的作用。



4.1研究背景与意义

四足机器人的发展趋势主要有以下四个方向：



目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

5.1 Ascend开发板下文件目录

六、精准控制 (4.5学时)

七、运行示例

八、附录

5.1 Ascend开发板下文件目录



5.1 Ascend开发板下文件目录

Command 文件夹

- ◆ **udp_command.py**: 该文件用于存放指令码。它定义了各种控制命令，这些命令通过UDP协议发送到机器狗，以实现对其精确控制。

SendCommand 文件夹

- ◆ **heartbeat.py**: 该文件用于发送心跳包。心跳包是一种定期发送的信号，用于检测系统或设备是否在线和正常工作。通过发送心跳包，可以确保机器狗与控制系统之间的通信连接是稳定和可靠的。
- ◆ **SendToCommand.py**: 该文件负责发送指令。它包含了一系列函数，用于将指令发送到机器狗，确保指令能够正确地传达和执行。

5.1 Ascend开发板下文件目录

SocketNetwork 文件夹

- ◆ **network_utils.py**: 该文件是与机器狗建立通信的脚本。它包含了各种网络通信的实用函数，用于初始化和维护与机器狗的通信连接，使得控制指令能够通过网络传输到机器狗。

Speeds 文件夹

- ◆ **sportspeed.py**: 该文件定义了机器狗运动的速度方程。通过调整速度参数，可以控制机器狗的运动速度，使其能够按照预定的轨迹和速度执行任务。

ThreadingUtils 文件夹

- ◆ **ThreadTemplates.py**: 该文件定义了线程模板。线程是并发执行的基本单位，通过使用线程，可以提高系统的响应速度和处理效率。该文件中的模板可以用于创建和管理多个线程，以实现并行处理。

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

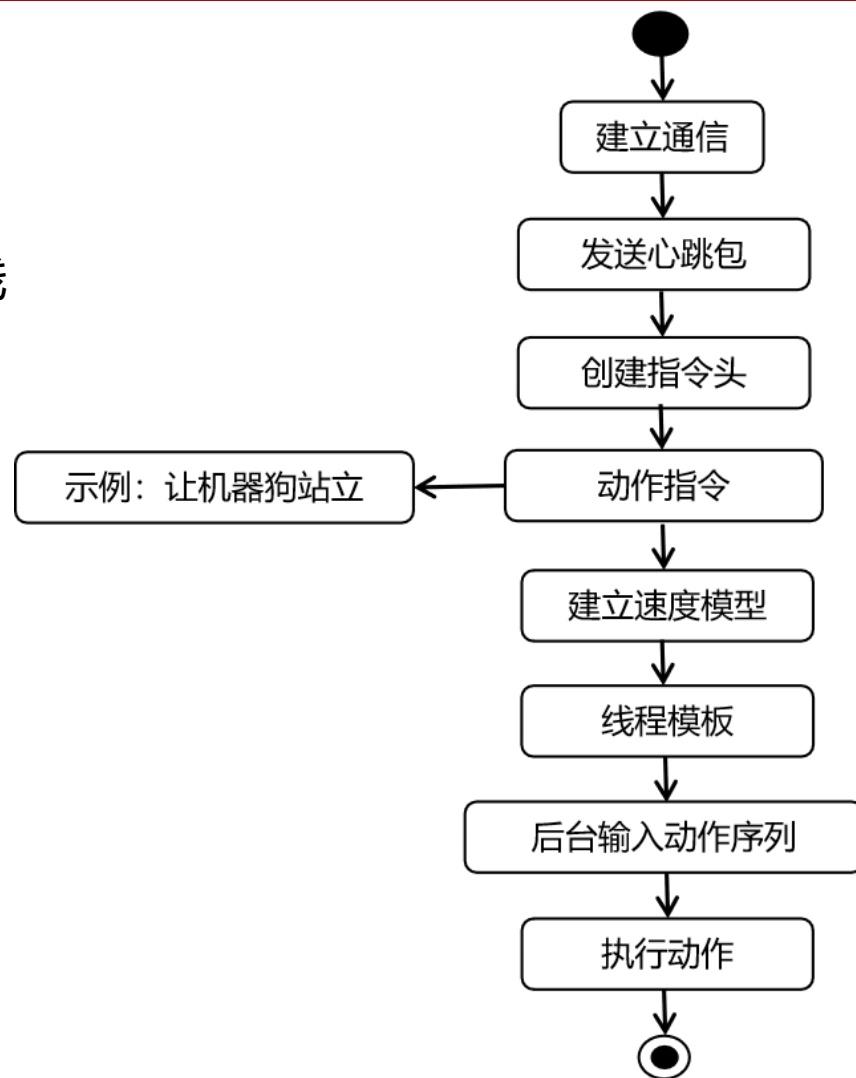
6.9执行动作

七、运行示例

八、附录

六、精准控制

第六章流程图分为 建立通信、发送心跳包、创建指令头、动作指令如发送指令使机器狗站立、建立速度模型、线程模板、后台输入动作序列、执行动作。



目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.1建立通信

简单控制第一步：建立通信链接， socketnetwork/network_utils.py文件定义了一个函数 setup_socket_and_address，用于创建一个UDP套接字，并设置机器狗的IP地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```
import socket
from typing import *
def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -
    > Tuple[socket.socket, Tuple[str, int]]:
    # 创建UDP套接字
    sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    # 设置目标地址
    target_address = (dest_ip, port)
    # print(target_address)
    return sfd, target_address
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.2发送心跳包

心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。发送心跳包代码位于 sendcommand/heartbeat.py，代码如下：

```
try:
    while True:
        # 记录发送前的时间
        start_time = time.time()

        # 发送心跳包
        sfd.sendto(heartbeat_command, target_address)

        # 发送后的延时
        time.sleep(max(0, heartbeat_interval - (time.time() - start_time)))

except KeyboardInterrupt:
    print("Heartbeat sending stopped by user.")
finally:
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

6.2发送心跳包

建立通讯

```
sfd, target_address = network_utils.setup_socket_and_address()
```

定义发送心跳包的线程执行体

*# 注意*args必须以tuple形式传入，包括sfd和target_address*

```
heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbe  
eat, 0.25, None, sfd, target_address)
```

```
heartbeat_thread.start()
```

启动心跳线程

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.3创建指令头

创建指令头主要为了构建通信协议，提高代码的可读性和可维护性以及支持复杂的指令操作。指令头包含了关键信息，如指令码、参数大小和类型，是通信协议的核心组成部分。通过定义指令头，可以构建和解析复杂的通信协议。

```
1. # 定义 CommandHead 类, 用于表示指令的头部信息
2. class CommandHead:
3.     # CommandHead 类的初始化方法
4.     def __init__(self, code=0, parameters_size=0, type_=0):
5.         # 指令码, 用于标识指令
6.         self.code = code
7.         # 指令参数大小, 表示指令参数所占的字节数
8.         self.parameters_size = parameters_size
9.         # 指令类型, 用于区分不同类型的指令
10.        self.type_ = type_
11.
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

6.3创建指令头

```
1. # 定义 Command 类, 用于表示一个完整的指令, 包括头部和数据
2. class Command:
3.     # 指令数据的大小常量, 设为 256 字节
4.     kDataSize = 256
5.     # Command 类的初始化方法
6.     def __init__(self, head=None, data=None):
7.         # 如果没有提供指令头, 则创建一个默认的 CommandHead 对象
8.         if head is None:
9.             head = CommandHead()
10.        # 如果没有提供数据, 则创建一个大小为 kDataSize 的全0列表
11.        if data is None:
12.            data = [0] * self.kDataSize
13.        # 将指令头和数据保存到实例变量
14.        self.head = head
15.        self.data = data
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.4动作指令

动作指令目的是调用机器狗的具体动作，通过将动作代码传入结构体中，并通过函数发送指令到机器狗，实现远程控制（dog_actions定义详见手册）

```
1. # 调用机器狗动作
2. # 设置 action_CommandHead 的 code 属性为 dog_actions 字典中 "ACTION_STAND_DOWN" 对应的值
3. # action_CommandHead 是一个 CommandHead 类的实例，用于存储动作的头部信息
4. # dog_actions 是一个字典，包含了不同动作的标识码
5. action_CommandHead.code = dog_actions.get("ACTION_STAND_DOWN")
6.
7. # 使用 SendToCommand 类的 perform_action 方法发送动作命令
8. # sfd 是一个套接字文件描述符，用于网络通信
9. # target_address 是目标地址，包含了 IP 地址和端口号
10. # action_CommandHead.code 是要发送的动作命令的代码
11. # 0, 0 是动作命令的额外参数，这里传入了 0
12. SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, 0, 0)
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.5示例：让机器狗站立

这是一个测试实例，通过简单的命令指令让机器狗站立，展示了如何通过简单的命令指令让机器狗站立。通过导入相关模块、建立通信链接、发送心跳包和发送站立指令，实现对机器狗的基本控制。为后续更复杂的操作提供了基础，验证了通信和控制指令的有效性。

导入定义的机器狗相关的包

```
from sendcommand import SendToCommand, heartbeat # 发送简单指令与心跳包
```

```
from socketnetwork import network_utils # 通信函数
```

```
from command.udp_command import * # 存放各种结构体与状态数据、指令
```

建立通讯

```
sfd, target_address = network_utils.setup_socket_and_address()
```

建立心跳包线程，按照心跳函数中的频率发送心跳包

```
heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
```

```
heartbeat_thread.start()
```

站立指令

```
SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.6建立速度模型

UDP指令下发之后，对于狗的位移距离是根据UDP下发多长时间决定，而在相同的时间内，val值的大小又会对狗的速度造成影响，所以没法精确控制狗的位移距离，如果是抽象的val值控制速度则更难，所以本实验提出了一种简单的思路来解决狗的控制问题，加入我们想要让狗位移5米，直接告诉他5米，他就会自己前进5米，这一切的基础是能得到狗的一种控制方式，这个方式叫做速度模型，下面给出速度模型的样例和构建思路。

long(m)	time(s)	每秒的平均	val
0.07	1	0.07	7000
0.12	2	0.06	7000
0.2	3	0.066666667	7000
0.1	1	0.1	7300
0.23	2	0.115	7300

0.33	3	0.11	7300
0.14	1	0.14	7400
0.28	2	0.14	7400
0.38	3	0.126666667	7400
0.15	1	0.15	7500
0.3	2	0.15	7500
0.45	3	0.15	7500
0.18	1	0.18	7700
0.38	2	0.19	7700

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

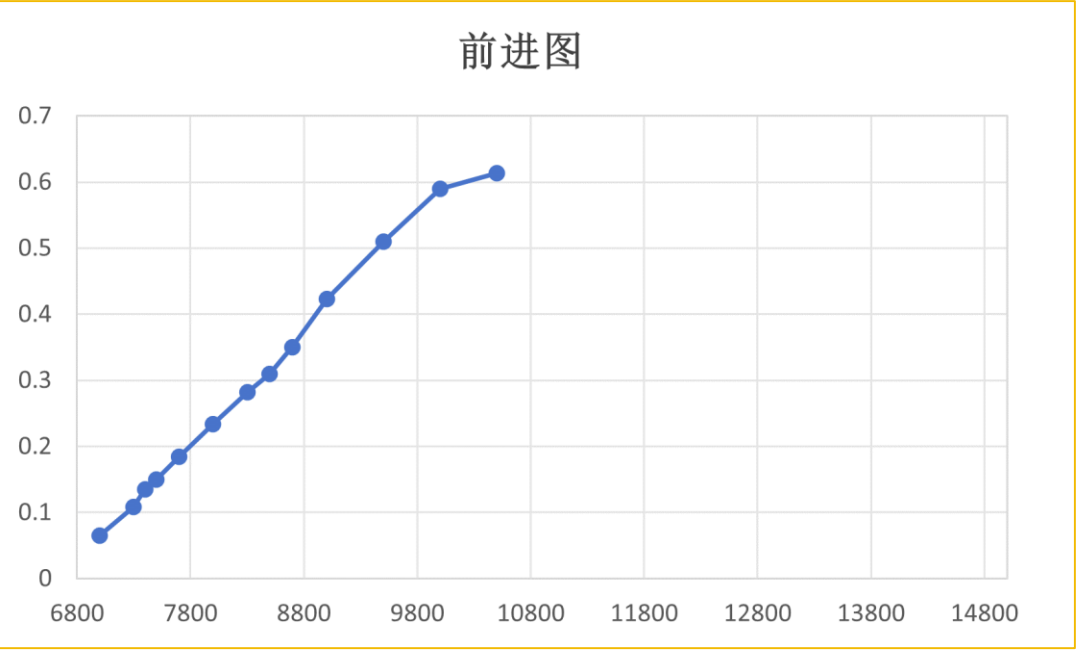
发送心跳包

建立通信

6.6建立速度模型

long(m)	time(s)	每秒的平均	val
0.07	1	0.07	7000
0.12	2	0.06	7000
0.2	3	0.06666667	7000
0.1	1	0.1	7300
0.23	2	0.115	7300
0.33	3	0.11	7300
0.14	1	0.14	7400
0.28	2	0.14	7400
0.38	3	0.12666667	7400
0.15	1	0.15	7500
0.3	2	0.15	7500
0.45	3	0.15	7500
0.18	1	0.18	7700
0.38	2	0.19	7700

上表是采集到的数据具体含义（以第一行为例）：在val值给出7000的情况下，UDP指令下发持续1秒钟，机器狗走了0.07m；后面的数据依次类推，然后对所有的值进行大量的测试，包括横向平移，选择，可以得到如下图像（给出前进的图像，其他方式与前进的测量方式完全相同，要注意一点，每个人测量的是不同）：



上图中y轴是速度，x轴是对应的val值，在对曲线进行拟合可以得到方程组。

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

6.6建立速度模型

```
1.  if long < 0:
2.      val = -val
3.      # 根据val计算速度 (单位是m/s) , 这里使用了二次方程
4.      speed_per_second_meter = (-1.5059e-08) * val ** 2 - (4.1944e-04) * val - 2.1804
5.      # 计算所需时间, 使用abs确保时间是正数
6.      times = abs(long / speed_per_second_meter)
7.      # 如果long参数大于等于0, 则使用另一组系数计算速度
8.  else:
9.      speed_per_second_meter = (-7.237e-09) * val ** 2 + 0.0002933 * val - 1.643
10.     # 计算所需时间
11.     times = long / speed_per_second_meter
12.     # 返回一个列表, 包含计算出的时间和速度
13.     return [times, val]
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.7线程模板

```
1. self.stopped.set() # 确保线程停止状态被设置
2. logging.info(f'离开线程: {self.name}')
3. # 记录日志, 表示线程已退出。
4. # 检查是否超过时间阈值并停止线程的方法
5. def check_time_and_stop(self, current_time) -> bool:
6.     # 定义一个方法来判断当前时间是否超过了线程开始时间加上时间阈值。
7.     if current_time - self.start_time > self.time_limit:
8.         logging.info(f'{self.name}由于超过时间阈值{self.time_limit}秒, 系统自动停止! ')
9.         # 如果当前时间减去线程开始时间大于时间阈值, 则记录日志并设置停止事件。
10.        self.stopped.set()
11.        return True
12.    elif self.global_var == 1:
13.        self.stopped.set()
14.        return True
15.    # 如果global_var变量为1, 也设置停止事件并返回True。
16.    return False
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

6.7线程模板

```
1. # 定义左右平移动作的函数, 返回一个线程实例
2. def action_turn_left_and_right(long, speedgear=3) -> MyRepeatThread:
3.     # 定义一个函数, 用于创建左右平移动作的线程实例。
4.     times, val = translate_left_and_right(long, speedgear)
5.     # 调用translate_left_and_right函数计算需要执行的次数和值。
6.     ACTION_turn_left_and_right_thread = MyRepeatThread(
7.         "ACTION_turn_left_and_right_thread",
8.         SendToCommand.perform_action, 0.1, times, sfd, target_address, 0x
9.         21010131, val, 0
10.    )
11.    # 创建并返回一个MyRepeatThread线程实例, 用于执行左右平移动作。
12.    # 不直接启动线程
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.8后台输入动作序列

后台输入动作序列主要实现了一个用于用户输入和处理机器狗动作序列的系统。用户通过输入一系列动作ID和相应的参数，系统将根据这些输入生成并更新动作序列，并提示用户为每个动作提供具体参数。

```
1. # 定义一个字典，将动作编号映射到相应的动作描述
2. actions_name = {
3.     1: '直走 (0.3表示前进0.3米, -0.3表示后退0.3米)',
4.     2: '平移 (0.3表示右平移0.3米, -0.3表示左平移0.3米)',
5.     3: '旋转 (45表示向右旋转45度, -45表示向左旋转45度)'
6. }
7. # 初始化一个空字典，用于存储动作参数
8. actions_params = {}
9. # 初始化一个空列表，用于存储动作序列
10. actions_sequence = []
11. # 初始化三个空列表，分别用于存储每个动作编号对应的参数值
12. val1, val2, val3 = [], [], []
13. print("下面即将进入到输入环节，根据你输入的数字和参数索引，机器狗将会做出相应运动")
14. print("输入格式：动作ID,参数索引，例如：3,2,1,2,3,1（依次用英文逗号分隔开）")
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

6.8后台输入动作序列

```
1. # 更新动作序列
2. actions_sequence = update_actions_sequence(actions_sequence)
3. # 提示用户可以设置速度挡位
4. logging.info('这里可以设置速度挡位, 比如在显示前进的时候可以输入 0.3,6 意思是以6档的速度前进0.3米')
5. # 遍历更新后的动作序列, 并打印每个动作的名称
6. for index, action in enumerate(actions_sequence):
7.     action_name = actions_name.get(action[0], "未知动作")
8.     print(f'第{index+1}个动作是: {action_name}')
9. # 接收用户对于每个动作的具体参数输入
10. for index, action in enumerate(actions_sequence):
11.     vals = input('你想要做的操作是: ')
12.     # 检查输入是否包含逗号
13.     if ',' in vals:
14.         # 如果包含逗号, 则将输入按逗号分割, 并转换为浮点数
15.         vals = tuple(float(val.strip()) for val in vals.split(','))
16.     else:
17.         # 如果没有逗号, 直接将输入转换为单个浮点数
18.         vals = (float(vals),)
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

6.1建立通信

6.2发送心跳包

6.3创建指令头

6.4动作指令

6.5示例：让机器狗站立

6.6建立速度模型

6.7线程模板

6.8后台输入动作序列

6.9执行动作

七、运行示例

八、附录

6.8执行动作

```
1. # 检查params是否为元组类型
2.     if isinstance(params, tuple):
3.         # 如果是元组, 调用动作函数并传入参数, 使用*操作符将元组中的元素作为独立的参数传递
4.         thread = action_func(*params) # 正确# 展开参数列表并传递给函数
5.         # 启动线程
6.         thread.start()
7.         # 等待线程执行完成
8.         thread.join()
9.         # 打印线程的属性信息, 这里假设thread对象有一个print_attributes方法
10.        pprint(thread.print_attributes())
11.        # 休眠1秒, 是为了在执行下一个动作之前提供短暂的延迟
12.        time.sleep(1)
13.    else:
14.        # 如果params不是元组或列表类型, 则记录错误日志
15.        logging.error('params 必须是一个 tuple 或者 列表')
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程简述
- 三、实验环境准备 (0.5学时)
- 四、机器狗介绍 (0.5学时)
- 五、工程文件目录
- 六、精准控制 (4.5学时)
- 七、运行示例
- 八、附录

7.1运行示例

如果要运行代码，直接输入

```
python precise_control_main.py
```

然后根据提示，输入相应的值即可，如下所示：

```
(base) root@davinci-mini:/opt/lab/PreciseControl# python precise_control_main.py
Starting HeartbeatThread
下面即将进入到输入环节，根据你输入的数字和参数索引，机械狗将会做出相应运动
输入格式：动作ID,参数索引，例如：3,2,1,2,3,1（依次用英文逗号分隔开）
请输入动作ID(回车结束)：1,2,3
INFO:root:这里可以设置速度档位，比如在显示前进的时候可以出入 0.3,6 意思是以6档的速度前进0.3米
第1个动作是：直走（0.3表示前进0.3米，-0.3表示后退0.3米）
你想要做的操作是：0.3
(0.3,)
第2个动作是：平移（0.3表示右平移0.3米，-0.3表示左平移0.3米）
你想要做的操作是：0.3
(0.3,)
第3个动作是：旋转（45表示向右旋转45度，-45表示向左旋转45度）
你想要做的操作是：30
(30.0,)
{1: [(0.3,)], 2: [(0.3,)], 3: [(30.0,)]}
INFO:root:-----
INFO:root:action_func: <function action_go_straight at 0xe7fff5498280>
INFO:root:params: (0.3,)
Starting ACTION_pan_back_and_forth_thread
INFO:root:ACTION_pan_back_and_forth_thread由于超过时间阈值1.2487928335941945秒，系统自动停止！
INFO:root:离开线程：ACTION_pan_back_and_forth_thread
INFO:root:-----
INFO:root:action_func: <function action_turn_left_and_right at 0xe7fff527fee0>
INFO:root:params: (0.3,)
Starting ACTION_turn_left_and_right_thread
INFO:root:ACTION_turn_left_and_right_thread由于超过时间阈值2.0866662029630656秒，系统自动停止！
INFO:root:离开线程：ACTION_turn_left_and_right_thread
INFO:root:-----
INFO:root:action_func: <function action_revolve_left_and_right at 0xe7fff527ff70>
INFO:root:params: (30.0,)
Starting ACTION_revolve_left_and_right
INFO:root:ACTION_revolve_left_and_right由于超过时间阈值0.2秒，系统自动停止！
INFO:root:离开线程：ACTION_revolve_left_and_right
```

（意思为第一步直走0.3米，第二步右平移0.3米，第三步右旋转30度）

目录

一、实验前置准备 (0.5学时)

二、实验过程简述

三、实验环境准备 (0.5学时)

四、机器狗介绍 (0.5学时)

五、工程文件目录

六、精准控制 (4.5学时)

七、运行示例

八、附录

8.1 腿部动力学建模

8.1 腿部动力学建模

拉格朗日方程

拉格朗日动力学方程是处理多自由度动力学问题的重要理论基础。对于处于完整约束并且约束力满足虚功原理的机械系统，可以使用拉格朗日动力学方程来描述系统随时间的变化。达朗贝尔原理指出，对于理想情况下具有两面约束的非自由质点系，在任一瞬时时刻，作用于该质系的主动动力及惯性力在质点系任意虚位移上所作的元功之和为零。可以表示为：

$$\sum_{i=1}^k (F_i - m_i \ddot{r}_i) \delta r_i = 0 \quad (2.1)$$

式中， r_i, m_i 分别表示质点 $P_i (i = 1, \dots, k)$ 的矢径和质量， F_i 则表示质点系统受到的主动力。

8.1 腿部动力学建模

由于虚拟约束 $\delta \mathbf{r}_i$ 并不相互独立，需要选择广义坐标来将方程(2.1))转换为关于独立广义坐标的表达式。
设 $q_j (j = 1, \dots, n)$ 为所研究质点的广义坐标，则可将矢径 $\mathbf{r}_i$ 表示成关于广义坐标的形式：

$$\delta \mathbf{r}_i = \sum_{j=1}^n \frac{\partial \mathbf{r}_i}{\partial q_j} \delta q_j \quad (2.2)$$

因此，根据式(2.1)，式(2.2)中左端第二项可变为可得：

$$\sum_{i=1}^k m_i \ddot{\mathbf{r}}_i \delta \mathbf{r}_i = \sum_{i=1}^k \sum_{j=1}^n m_i \ddot{\mathbf{r}}_i \frac{\partial \mathbf{r}_i}{\partial q_j} \delta q_j \quad (2.3)$$

使用微分运算可将上式重新整理得到：

$$\sum_{i=1}^k m_i \ddot{\mathbf{r}}_i \frac{\partial \mathbf{r}_i}{\partial q_j} = \sum_{i=1}^k \frac{d}{dt} \left(m_i \dot{\mathbf{r}}_i \frac{\partial \mathbf{r}_i}{\partial q_j} \right) - \sum_{i=1}^k m_i \dot{\mathbf{r}}_i \frac{d}{dt} \left(\frac{\partial \mathbf{r}_i}{\partial q_j} \right) \quad (2.4)$$

8.1 腿部动力学建模

接下来，根据公式：

$$\frac{\partial \dot{\mathbf{r}}_i}{\partial \dot{q}_j} = \frac{\partial \mathbf{r}_i}{\partial q_j} \quad (2.5)$$

$$\frac{d}{dt} \left(\frac{\partial \mathbf{r}_i}{\partial q_j} \right) = \frac{\partial \dot{\mathbf{r}}_i}{\partial q_j} \quad (2.6)$$

$$K = \sum_{i=1}^k \frac{1}{2} m_i \dot{\mathbf{r}}_i^2 \quad (2.7)$$

式(2.7)可变为：

$$\sum_{i=1}^k m_i \ddot{\mathbf{r}}_i \frac{\partial \mathbf{r}_i}{\partial q_j} = \frac{d}{dt} \frac{\partial K}{\partial \dot{q}_j} - \frac{\partial K}{\partial q_j} \quad (2.8)$$

将式(2.8)带入式(2.1)可得：

$$\sum_{j=1}^n \left(\sum_{i=1}^k \mathbf{F}_i \frac{\partial \mathbf{r}_i}{\partial q_j} - \frac{d}{dt} \frac{\partial K}{\partial \dot{q}_j} + \frac{\partial K}{\partial q_j} \right) \delta q_j = 0 \quad (2.9)$$

8.1 腿部动力学建模

由于广义坐标相互独立，同时令第 j 个广义力为：

$$\psi_j = \sum_{i=1}^k F_i \frac{\partial r_i}{\partial q_j} \quad (2.10)$$

引入势能函数 $P(q)$ ，则外界施加的广义力可表示为主动动力 F_i 引起的广义力 ψ_j 与势能 $P(q)$ 引起的广义力之和：

$$\tau_j = \psi_j + \frac{\partial P(q)}{\partial q_j} \quad (2.11)$$

根据式子(2.10)和(2.11)，公式(2.9)可被写为下列形式：

$$\frac{d}{dt} \left(\frac{\partial \mathcal{L}}{\partial \dot{q}_j} \right) - \frac{\partial \mathcal{L}}{\partial q_j} = \tau_j \quad (2.12)$$

8.1 腿部动力学建模

式中， $\mathcal{L} = K - P$ 为拉格朗日算子,定义为系统的动能减去势能，化简后得到：

$$\frac{d}{dt} \left(\frac{\partial K}{\partial \dot{q}_j} \right) - \frac{\partial K}{\partial q_j} + \frac{\partial P}{\partial q_j} = \tau_j \quad (2.13)$$

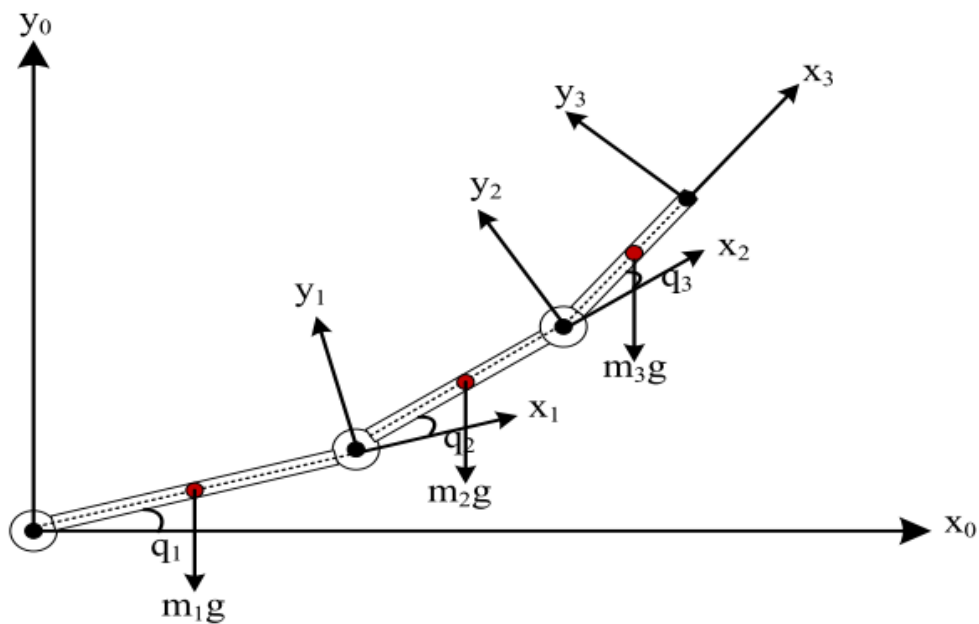
式(2.13)为拉格朗日动力学方程。

8.1 腿部动力学建模

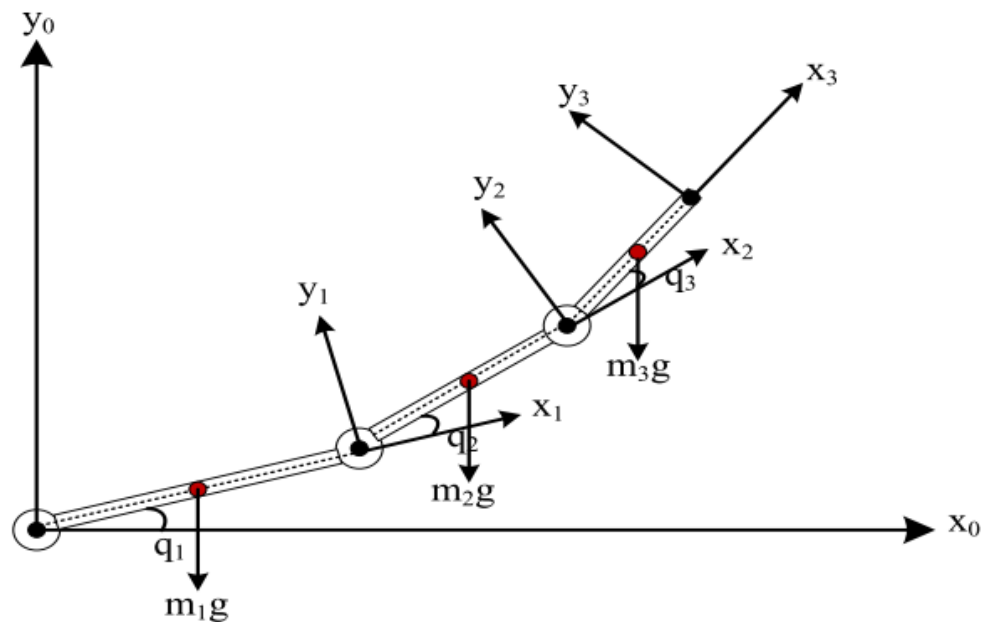
将拉格朗日方程写成腿的动力学方程的表达式:

$$M(q)\ddot{q} + C(q, \dot{q}) + G(q) = \tau$$

在三关节手指建模中, $\tau = [\tau_1 \quad \tau_2 \quad \tau_3]^T$ 是控制力的矩阵, $\ddot{q} = [\ddot{q}_1 \quad \ddot{q}_2 \quad \ddot{q}_3]^T$ 是角加速度向量, $M(q) \in \mathbb{R}^{3 \times 3}$ 为惯性矩阵, $C(q, \dot{q}) \in \mathbb{R}^{3 \times 1}$ 是哥式力和离心力的结合向量, $G(q) \in \mathbb{R}^{3 \times 1}$ 是重力矩阵



8.1 腿部动力学建模



图中，对于 $i = 1, 2, 3$ ， q_i 表示关节转角，也作为广义坐标； m_i 表示手指指节的质量； l_i 表示指节的长度； l_{ci} 表示前一个关节与指节 i 的质心之间的距离。

此处使用 Denavit-Hartenberg 关节变量作为广义坐标，从而可得到表达式：

$$v_n^0 = J_v \dot{q} \quad (2.14)$$

$$w_n^0 = J_w \dot{q}$$

8.1 腿部动力学建模

式中， v_n^0 代表灵巧手指质心的线速度， w_n^0 代表机器狗指质心角速度， n 代表关节数量。 J_v 表示各关节对应的雅可比矩阵， $\dot{q}$ 表示各关节转动的角速度。

根据柯尼希定理，动能等于杆件质心的平动动能和杆件相对于质心的转动动能之和，则系统总动能

$$K = \frac{1}{2} m v^T v + \frac{1}{2} \omega^T I \omega$$

$$I = R I R^T$$

I 是附体坐标系的惯性张量矩阵， I 是惯性坐标系中的惯性张量矩阵，将（下面式子带入上面式子）得到：

$$K = \frac{1}{2} \dot{q}^T \left[\sum_{i=1}^n \{ m_i J_{vi}^T J_{vi} + J_{\omega i}^T R_i I_i R_i^T J_{\omega i} \} \right] \dot{q} = \frac{1}{2} \dot{q}^T M \dot{q}$$

得到 M 为：

$$M(q) = \begin{bmatrix} m_{11} & m_{12} & m_{13} \\ m_{21} & m_{22} & m_{23} \\ m_{31} & m_{32} & m_{33} \end{bmatrix}$$

8.1 腿部动力学建模

科里奥利矩阵经过计算可得：

$$c_{kj} = \sum_{i=1}^n c_{ijk}(q) \dot{q}_i = \sum_{i=1}^n \frac{1}{2} \left\{ \frac{\partial m_{kj}}{\partial q_i} + \frac{\partial m_{ki}}{\partial q_j} - \frac{\partial m_{ij}}{\partial q_k} \right\} \dot{q}_i$$

$$c_{ijk} = \frac{1}{2} \left\{ \frac{\partial d_{ki}}{\partial q_j} + \frac{\partial d_{kj}}{\partial q_i} - \frac{\partial d_{ij}}{\partial q_k} \right\}$$

将下面式子带入上面式子得到：

$$C(q, \dot{q}) = \begin{bmatrix} c_{11} & c_{12} & c_{13} \\ c_{21} & c_{22} & c_{23} \\ c_{31} & c_{32} & c_{33} \end{bmatrix}$$

8.1 腿部动力学建模

灵巧手指的重力势能等于三根指节的势能之和，通过每根指节质量乘以重力加速度以及质心高度计算得到：

$$P = P_1 + P_2 + P_3$$

通过对重力势能微分可得重力向量矩阵：

$$g(q) = \begin{bmatrix} g_1 \\ g_2 \\ g_3 \end{bmatrix}$$

因此，由以上公式可推导出机器狗腿动力学模型为：

$$M(q)\ddot{q} + C(q, \dot{q}) + G(q) = \tau$$

8.1 腿部动力学建模

系统动力学模型简化为：

$$\ddot{q} = D(q)^{-1}(\tau - C(q, \dot{q})\dot{q} - g(q))$$

令 $x_1(t) = q, x_2(t) = \dot{q}$ ，那么机器狗腿动力学模型状态空间表达式表示为：

$$\begin{cases} \dot{x}_1(t) = x_2(t) \\ \dot{x}_2(t) = D(x_1(t))^{-1}(\tau - C(x_1(t), x_2(t))x_2(t) - g(x_1(t))) \\ y(t) = x_1(t) \end{cases}$$

感谢

版权所有©2024，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编、也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。

华为开发者创新中心

<https://connect.huaweicloud.com>